

Project 3

Harry Hawkins 48019675 - 2024

[Git repo here](#)

```
In [4]: using Pkg
        Pkg.activate(".")
```

```
Activating project at `C:\Users\hawki\Uni\M2504\P3\Harry-Hawkins-2504-2024-PROJECT3`
```

General data observations

In tasks 1-3 as the purpose is exploring data, effectively all missing values will be skipped to get accurate interpretations of the relationships the data can supply. Altering missing values by averages of any type will influence the data plots and cause innacuracy in the models approximaated from the data.

```
In [5]: using CSV, DataFrames, HTTP, CSV, Plots, StatsPlots, StatsBase, Dates, SplitAppl
        path = "./data/Melbourne_housing_FULL.csv"
        old_df = CSV.read(path, DataFrame)
        println("collected data")
```

collected data

Note some of the column names and what they represent taken from the data source:

Propertycount: Number of properties that exist in the suburb.

Distance: Distance from CBD in Kilometres

Method:

S - property sold;

SP - property sold prior;

PI - property passed in;

PN - sold prior not disclosed;

SN - sold not disclosed;

NB - no bid;

VB - vendor bid;

W - withdrawn prior to auction;

SA - sold after auction;

SS - sold after auction price not disclosed.

N/A - price or highest bid not available.

Type:

br - bedroom(s);

h - house,cottage,villa, semi,terrace;

u - unit, duplex;
t - townhouse;
dev site - development site;
o res - other residential.

SellerG: Real Estate Agent

Bedroom2 : Scraped # of Bedrooms (from different source)

I will check what rows have missing values or how missing values are represented and if they are not represented as missing values, they will be changed to be missing. Some missing values can be turned into other values such as -1 or 0 if the data does not need to be used in the exploratory phases of the tasks. For columns related to areas, I will replace data by going through other columns that have the relevant data. Columns datatypes may also be changed to ensure numbers are represented as a number type, e.g. date column represented as Date type.

```
In [6]: df = copy(old_df)
# unique(df.Suburb) #No missing
# unique(df.Address) #No missing
# unique(df.Rooms) #No missing
# unique(df.Type) #No missing
# unique(df.Method) #No missing
# unique(df.SellerG) #No missing
# unique(df.Date) #No missing

# unique(df.Price) #missing as missing => will need to skip or avg

# unique(df.Distance) #missin as #N/A
allowmissing!(df, :Distance)
for (i,n) in enumerate(df.Distance)
    if n == "#N/A"
        df.Distance[i] = missing
    end
end
df.Distance .= passmissing(parse).(Float64, df.Distance)

# unique(df.Landsize) #missing as missing there is also zero values
# these zero values will be tested for properties with expected zero Landsize:
count = 0
for (index, val) in enumerate(skipmissing(df.Landsize))
    if val == 0 && (df.Type == "h" || df.Type == "t" || df.Type == "o res")
        count += 1
    end
end
# println(count) #This is 0
#Hence the only times the Landsize is zero is for units

# unique(df.YearBuilt) #missing as missing => -1
df.YearBuilt .= coalesce.(df.YearBuilt, -1)

# unique(df.Lattitude) #missing as missing => keep as
# unique(typeof.(df.Longtitude)) #missing as missing => keep as
```

```

# unique(df.Bedroom2) #missing as missing => keep as
# unique(df.Bathroom) #missing as missing => keep as
# unique(df.Car) #missing as missing => keep as

allowmissing!(df, :CouncilArea)
allowmissing!(df, :Postcode)
allowmissing!(df, :Regionname)
allowmissing!(df, :Propertycount)

# unique(df.CouncilArea) #missing as #N/A => changing based on suburb/address or
function findarea(suburb::String31)
    for (i,n) in enumerate(df.Suburb)
        if n == suburb && df.CouncilArea[i] != "#N/A"
            return df.CouncilArea[i]
        end
    end
    return missing
end

for (i,n) in enumerate(df.CouncilArea)
    if n == "#N/A" && !(df.Suburb[i] isa Nothing)
        df.CouncilArea[i] = findarea(df.Suburb[i])
    end
end

# unique(df.Postcode) #missing as #N/A => changing based on suburb/address
function findcode(suburb::String31)
    for (i,n) in enumerate(df.Suburb)
        if n == suburb && df.Postcode[i] != "#N/A"
            return df.Postcode[i]
        end
    end
    return missing
end

for (i,n) in enumerate(df.Postcode)
    if n == "#N/A"
        df.Postcode[i] = findcode(df.Suburb[i])
    end
end

# unique(df.Regionname) #missing as #N/A => changing based on suburb/address
function findregion(suburb::String31)
    for (i,n) in enumerate(df.Suburb)
        if n == "suburb" && df.Regionname[i] != "#N/A"
            return df.Regionname[i]
        end
    end
    return missing
end

for (i,n) in enumerate(df.Regionname)
    if n == "#N/A"
        df.Regionname[i] = findregion(df.Suburb[i])
    end
end

```

```

# unique(df.Propertycount) #missing as #N/A => changing based on suburb/address
function findcount(suburb::String31)
    for (i,n) in enumerate(df.Suburb)
        if n == "suburb" && df.Propertycount[i] != "#N/A"
            return df.Propertycount[i]
        end
    end
    return missing
end

for (i,n) in enumerate(df.Propertycount)
    if n == "#N/A"
        df.Propertycount[i] = findcount(df.Suburb[i])
    end
end

df.Propertycount = passmissing(parse).(Int64, df.Propertycount)
df.Postcode .= passmissing(parse).(Int64, df.Postcode)

# turns Date string into a form that can be turned into a Date type
function parsedate(date)
    result = ""
    for n in date
        if n == '/'
            result *= "-"
        else
            result *= n
        end
    end
    return result
end

#Creates the Date column to be a Date type
insertcols!(df, :newDate => Date("0001-01-01"))
for (i,n) in enumerate(df.Date)
    date = String(parsedate(n))
    df.newDate[i] = Date(date, "d-m-y")
end
df = select!(df, Not(:Date))
rename!(df, :newDate => :Date)

insertcols!(df, :newmeth => "")
for (i,n) in enumerate(df.Method)
    method = String(n)
    df.newmeth[i] = method
end
df = select!(df, Not(:Method))
rename!(df, :newmeth => :Method)

println("Missing values & data types changed")

```

Missing values & data types changed

Task 1

```

In [7]: typeof(df.Rooms) # Int64
typeof(df.Price) # Int64
typeof(df.Method) #String

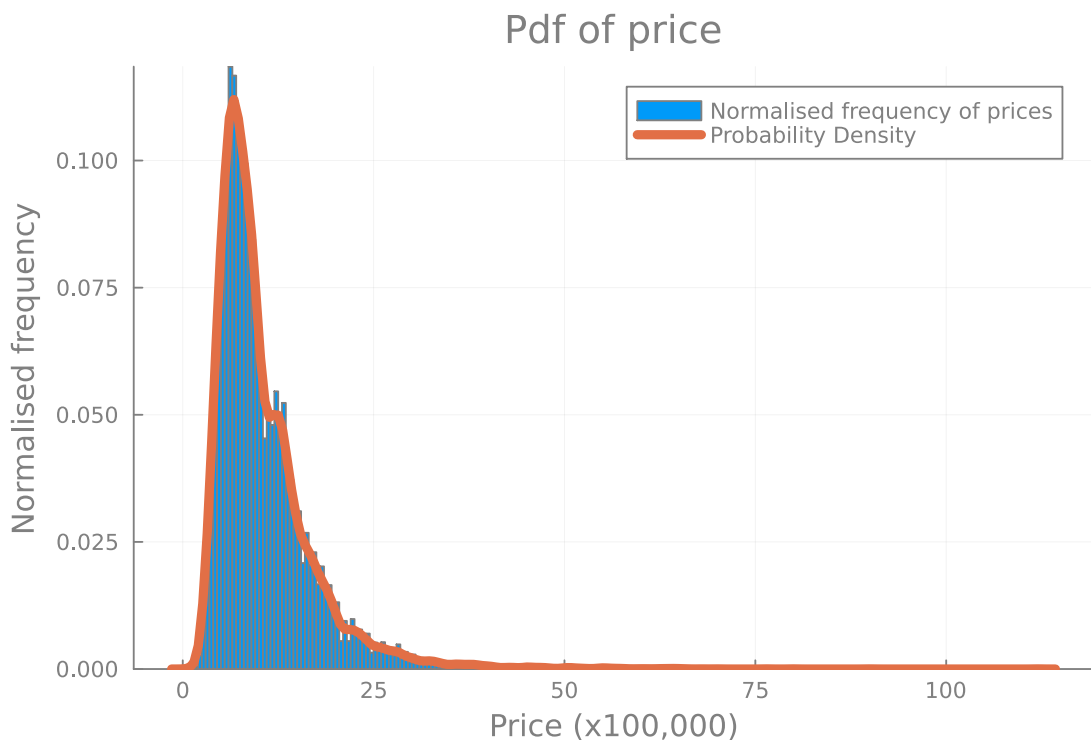
```

```
typeof(df.Distance) # Float64
typeof(df.Landsize) # Int64
default(foreground_color=:grey);
#Find distributions of each variable
#skip missing data in these plots as then averages medians can be found from the
```

```
In [8]: @show summarystats(df.Price)
plot1 = histogram(skipmissing(df.Price)./10^5, normalize=:pdf,
title = "Pdf of price",
xlabel = "Price (x100,000)",
ylabel = "Normalised frequency",
label = "Normalised frequency of prices",
)
density!(plot1, skipmissing(df.Price)./10^5, lw=5, label = "Probability Density")
```

```
summarystats(df.Price) = Summary Stats:
Length:      34857
Missing Count: 7610
Mean:        1050173.344955
Std. Deviation: 641467.130105
Minimum:      85000.000000
1st Quartile: 635000.000000
Median:       870000.000000
3rd Quartile: 1295000.000000
Maximum:      11200000.000000
```

Out[8]:



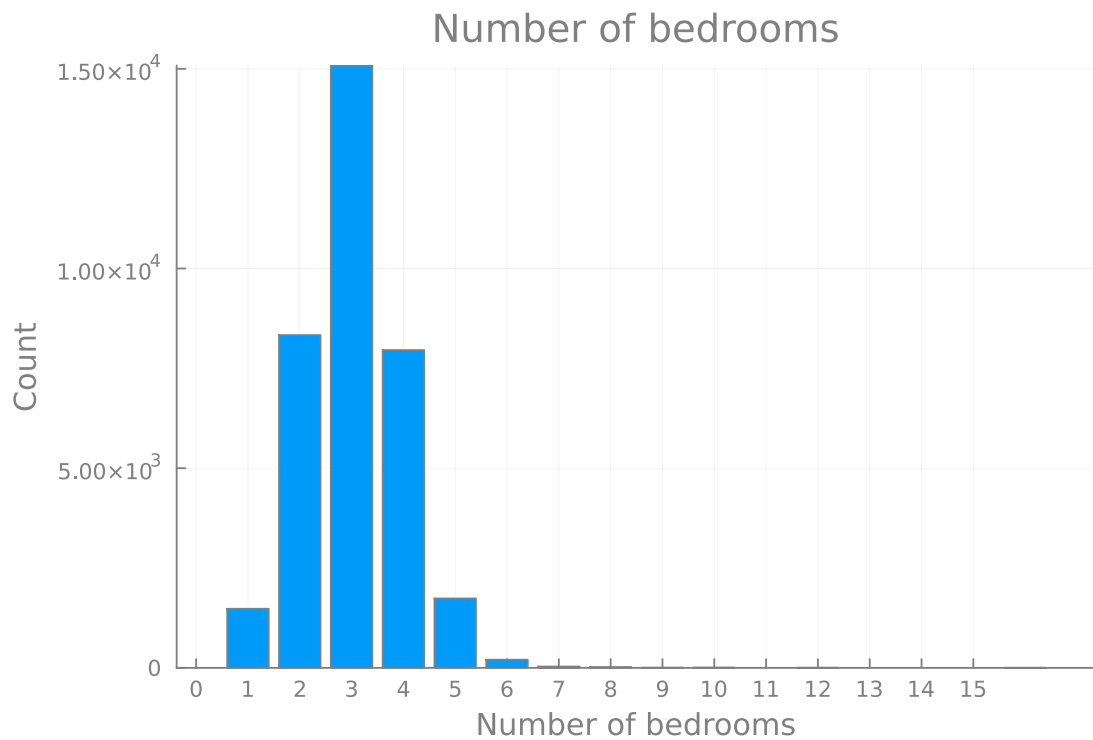
The pricing has a range of 85,000–11,200,000 with a mean of \$1,050,173.

```
In [9]: @show summarystats(df.Rooms)
#Rooms
grdf = groupcount(df.Rooms)
#From observing the DataFrame the number of rooms 7,8,9,10,12,16 are so small th
bar(collect(keys(grdf)), collect(values((grdf))),
title = "Number of bedrooms",
xlabel = "Number of bedrooms",
ylabel = "Count",
```

```
legend = false,  
xticks = 0:1:15)
```

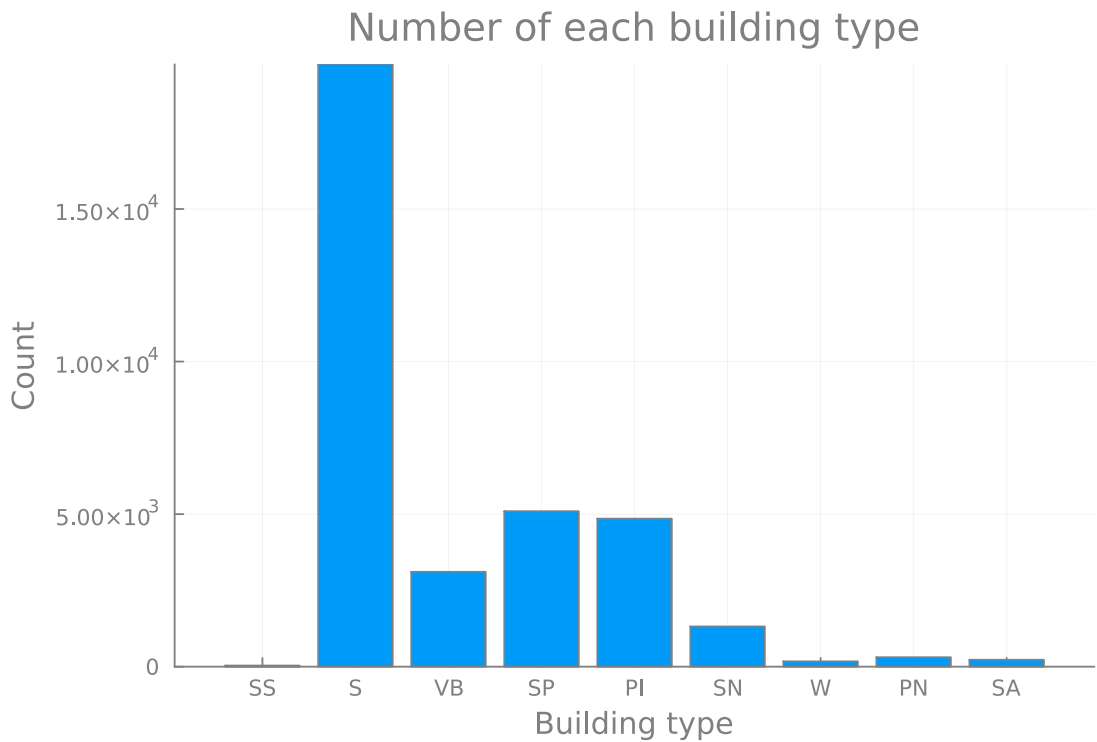
```
summarystats(df.Rooms) = Summary Stats:  
Length:      34857  
Missing Count: 0  
Mean:        3.031012  
Std. Deviation: 0.969933  
Minimum:     1.000000  
1st Quartile: 2.000000  
Median:      3.000000  
3rd Quartile: 4.000000  
Maximum:     16.000000
```

Out[9]:



```
In [10]: #Method  
gmdf = groupcount(df.Method)  
#SS has count 36  
bar(collect(keys(gmdf)), collect(values((gmdf))),  
title = "Number of each building type",  
xlabel = "Building type",  
ylabel = "Count",  
legend = false)
```

Out[10]:



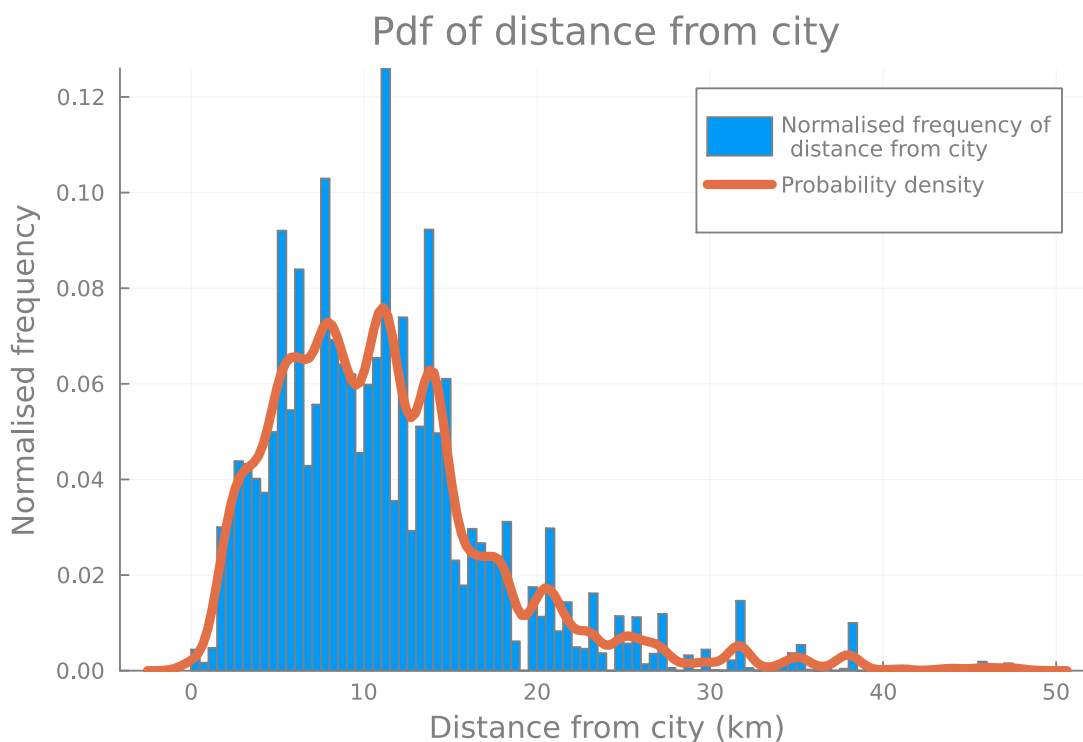
The most popular selling method is simply being that the property was sold in a standard way, 19,744, followed by being sold prior, 5095, and then property being passed in, 4850. However there were still many vendor bids and not disclosed selling at 3108 and 1317 respectively.

```
In [11]: @show summarystats(df.Distance)
#Distance
histogram(skipmissing(df.Distance),normalize=:pdf,
title = "Pdf of distance from city",
xlabel = "Distance from city (km)",
ylabel = "Normalised frequency",
label = "Normalised frequency of\n distance from city")
density!(skipmissing(df.Distance), lw=5, norm=true, label="Probability density")
```

```
summarystats(df.Distance) = Summary Stats:
```

```
Length:      34857
Missing Count: 1
Mean:        11.184929
Std. Deviation: 6.788892
Minimum:     0.000000
1st Quartile: 6.400000
Median:      10.300000
3rd Quartile: 14.000000
Maximum:     48.100000
```

Out[11]:



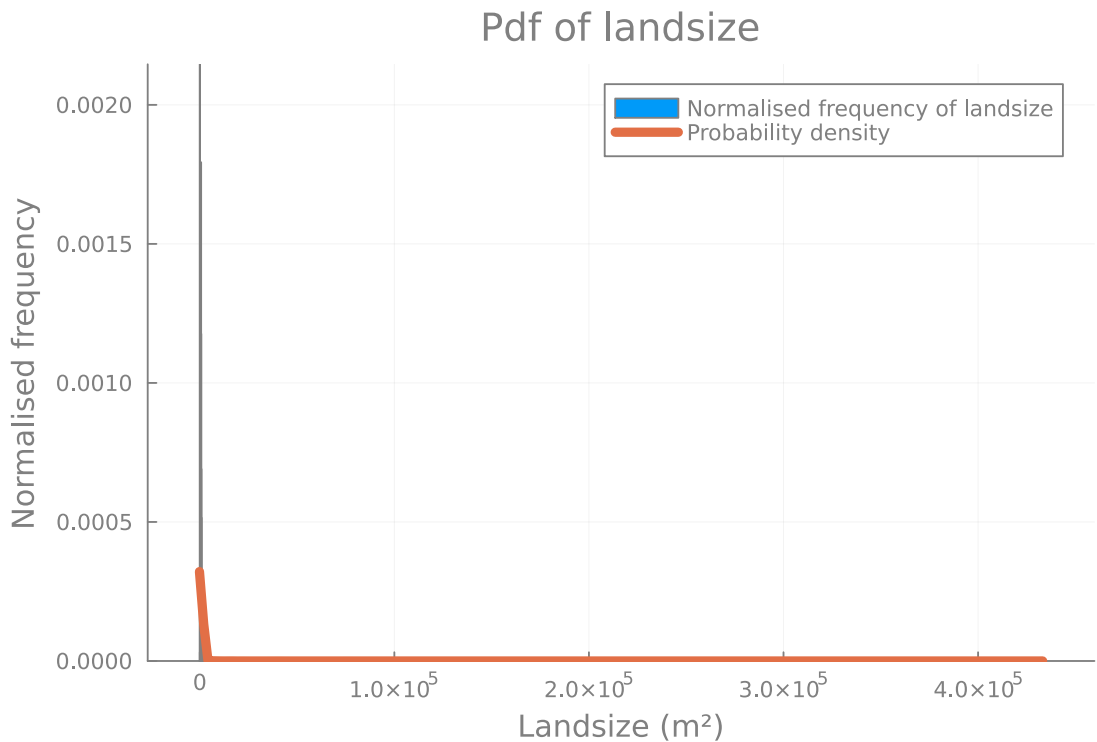
This data has a greater standard deviation than the price, but is still left oriented. With only one missing value, the data is favourable to test compared to other variables.

```
In [12]: @show summarystats(df.Landsize)
#Landsize
data = select(df, [:Landsize, :Type])

histogram(skipmissing(data.Landsize), normalize=:pdf,
title = "Pdf of landsize",
xlabel = "Landsize (m²)",
ylabel = "Normalised frequency",
label = "Normalised frequency of landsize")
density!(skipmissing(data.Landsize), lw=5, norm = true, label="Probability densi

summarystats(df.Landsize) = Summary Stats:
Length:      34857
Missing Count: 11810
Mean:        593.598993
Std. Deviation: 3398.841946
Minimum:     0.000000
1st Quartile: 224.000000
Median:      521.000000
3rd Quartile: 670.000000
Maximum:     433014.000000
```

Out[12]:



There are outliers that stretch the x axis so that the majority of data cannot be seen. The pdf could be approximated as exponential. For a clearer view of the majority of the data, I will observe the data limited to a landsize of 3000, just under the standard deviation. Also units will be omitted to better the value of data as all units will have a landsize of zero.

In [13]:

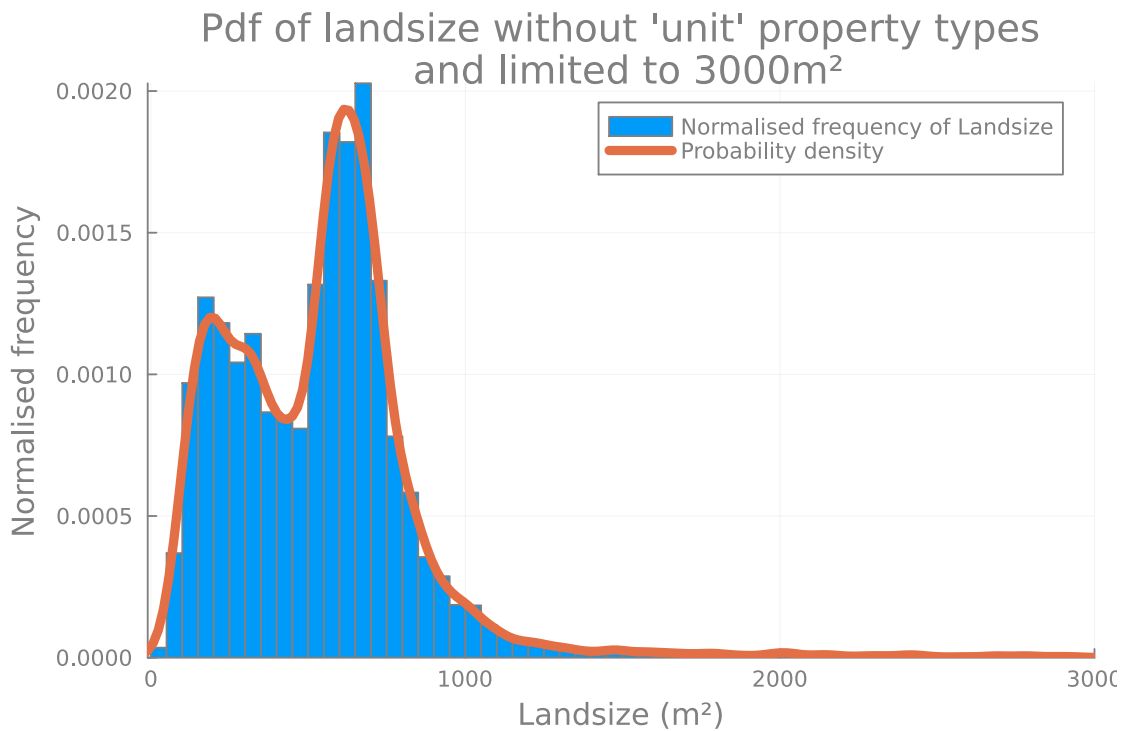
```
#Landsize without units
data = transform(data, :Type => x -> (x != "u"), :Landsize => y -> coalesce(y, -
data[!, :Landsize_function] = convert.(Int64, data[!, :Landsize_function])
landsizes = data.Landsize_function[data.Type_function .== true .&& data.Landsize
@show summarystats(landsizes)

histogram(landsizes, normalize=:pdf,
title = "Pdf of landsize without 'unit' property types\n and limited to 3000m²",
xlabel = "Landsize (m²)",
ylabel = "Normalised frequency",
label = "Normalised frequency of Landsize")
density!(landsizes, lw=5, norm = true, xlims=[-10, 3*10^3], label="Probability d
```

summarystats(landsizes) = Summary Stats:

Length:	20381
Missing Count:	0
Mean:	536.309259
Std. Deviation:	310.945435
Minimum:	1.000000
1st Quartile:	304.000000
Median:	554.000000
3rd Quartile:	685.000000
Maximum:	2978.000000

Out[13]:



More useful data can now be observed. Omitting units now makes the minimum value 1, Although this space is staggeringly small it might be some properties counting decks or verandas.

Task 2

When comparing the building area and landsize, the data has to be limited to values under 3000 to get a more accurate relation as there are some 'outliers' that stretch the x axis.

```
In [14]: #skip missing for price, distance, building area

#Price vs Distance
#price vs house size
# - Land
# - room
# - car
# - Bathroom
# - building area

#combine distance, area, Land - each are normally distributed continuous data so
#sum of rooms
data = select(df, [:Distance, :Landsize, :Rooms, :Car, :Bathroom, :BuildingArea],
sort!(data)
data = coalesce.(data, -1)
disallowmissing!(data);
```

```
In [15]: #price vs Distance
data = select(df, [:Price, :Distance])
data = groupby(data, :Distance)
data = combine(data, :Price => mean ∘ skipmissing => :Price)

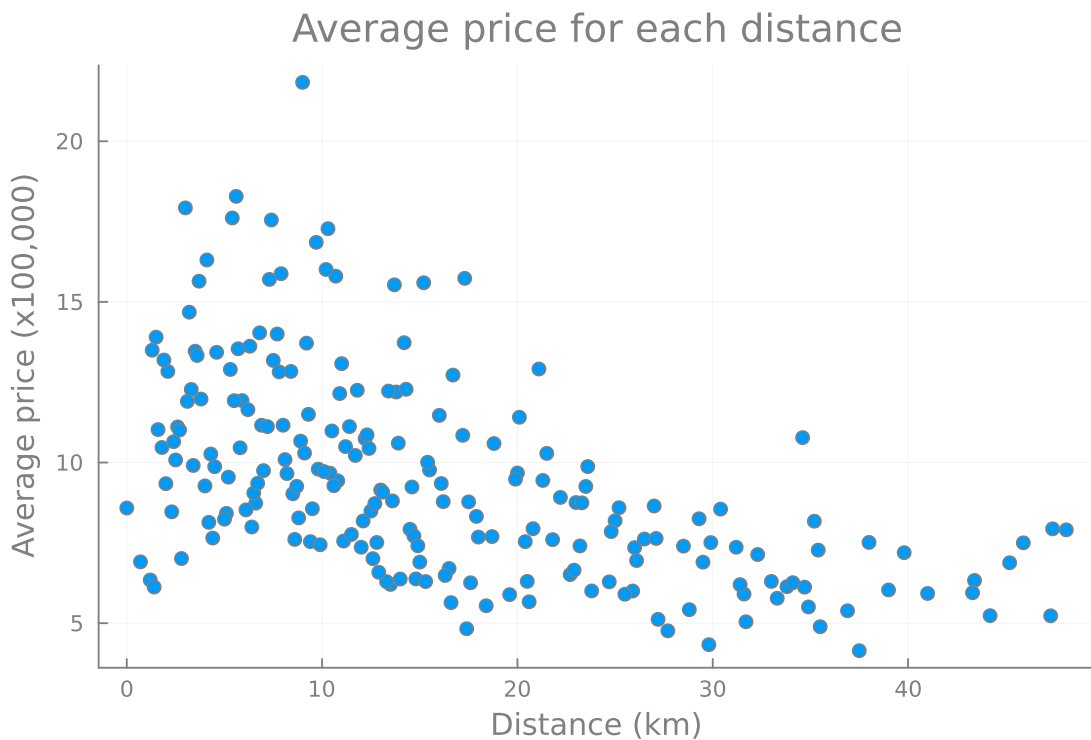
scatter(data.Distance, data.Price./10^5,
```

```

title = "Average price for each distance",
xlabel = "Distance (km)",
ylabel = "Average price (x100,000)",
legend = false)

```

Out[15]:



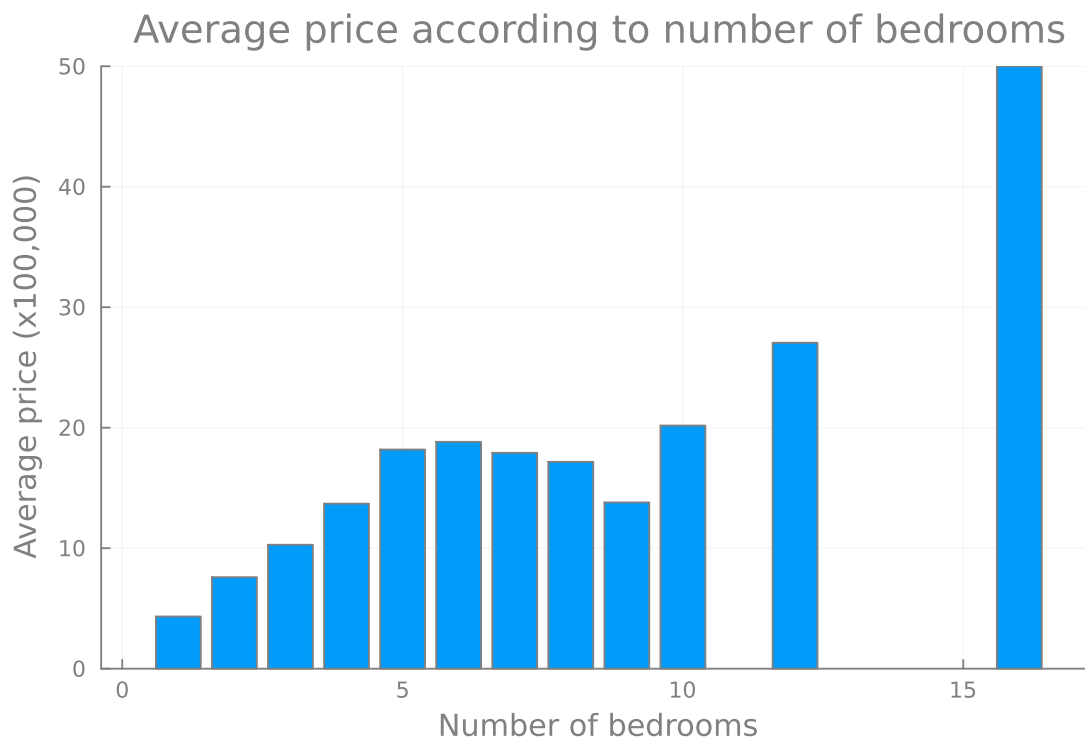
The relationship could be exponential decay. This makes sense as properties in the city are usually more expensive. Alternatively it could be polynomial as the data appears to be lower to begin with.

```

In [16]: #room
data = select(df, [:Price, :Rooms])
data = groupby(data, :Rooms)
data = combine(data, :Price => mean ◊ skipmissing => :Price)
bar(data.Rooms, data.Price./10^5,
title = "Average price according to number of bedrooms",
xlabel = "Number of bedrooms",
ylabel = "Average price (x100,000)",
legend = false)

```

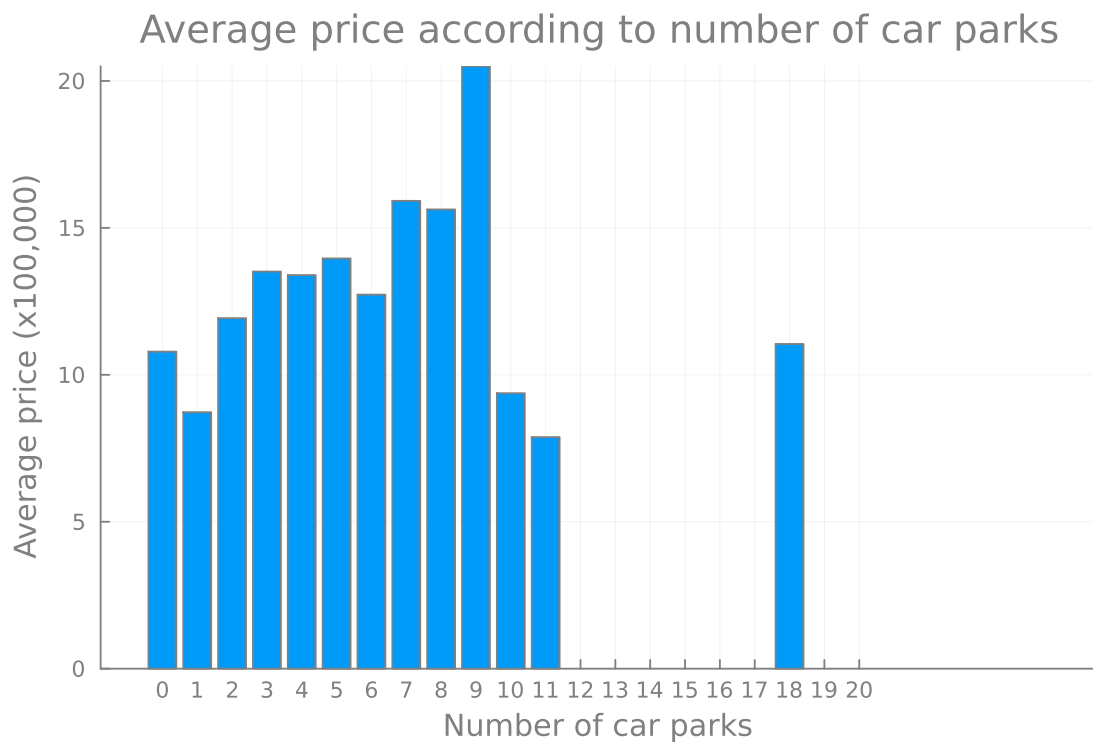
Out[16]:



This data could be approximated as cubic as it appears properties with 5 - 9 rooms go for the same average price. As expected The more rooms the more expensive as seen by the one property with a reported 16 rooms.

```
In [17]: #car
data = select(df, [:Price, :Car])
data = groupby(data, :Car)
data = combine(data, :Price => mean ◦ skipmissing => :Price)
bar(data.Car, data.Price./10^5,
title = "Average price according to number of car parks",
xlabel = "Number of car parks",
ylabel = "Average price (x100,000)",
xticks = 0:1:20,
legend = false)
```

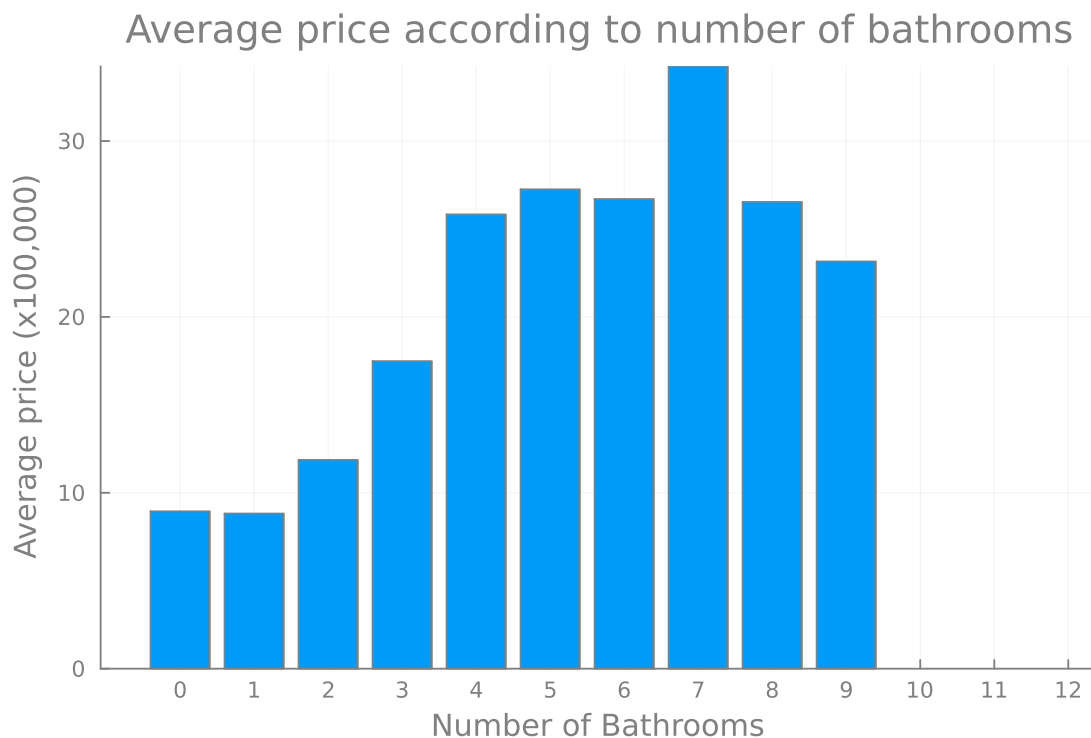
Out[17]:



There are up to 26 car spots in the recorded data, but due to missing price values, they are not displayed. Furthermore, if not just for missing values, the sudden decreases in average price may be explained if sellers of units advertise the size of underground parking.

```
In [18]: #bathroom
data = select(df, [:Price, :Bathroom])
data = groupby(data, :Bathroom)
data = combine(data, :Price => mean ◦ skipmissing => :Price)
bar(data.Bathroom, data.Price./10^5,
title = "Average price according to number of bathrooms",
xlabel = "Number of Bathrooms",
ylabel = "Average price (x100,000)",
xticks = 0:1:12,
legend = false)
```

Out[18]:

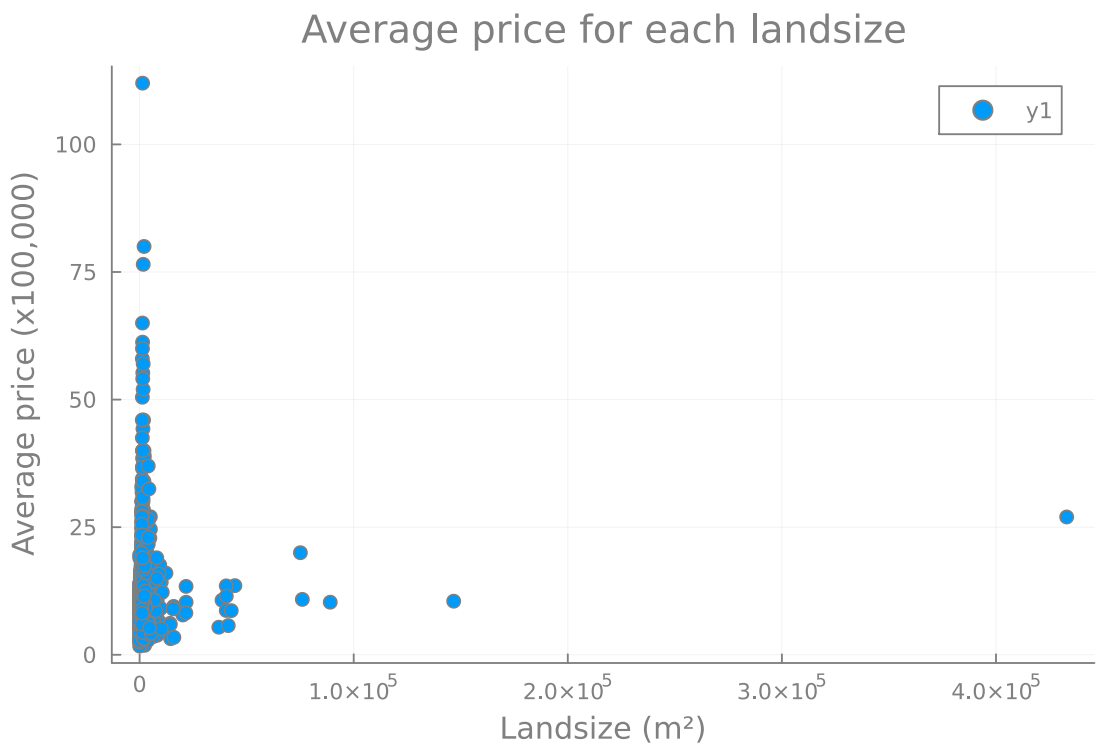


This appears as though it will become normally distributed, but it is more likely there are missing price values in the higher numbers. However, the more bathrooms the higher the price the relation for most of this data.

In [19]:

```
#Landsize
data = select(df,
[:Price, :Landsize])
data = groupby(data, :Landsize)
data = combine(data, :Price => mean ◦ skipmissing => :Price)
scatter(data.Landsize, data.Price./10^5,
title = "Average price for each landsize",
xlabel = "Landsize (m²)",
ylabel = "Average price (x100,000)")
```

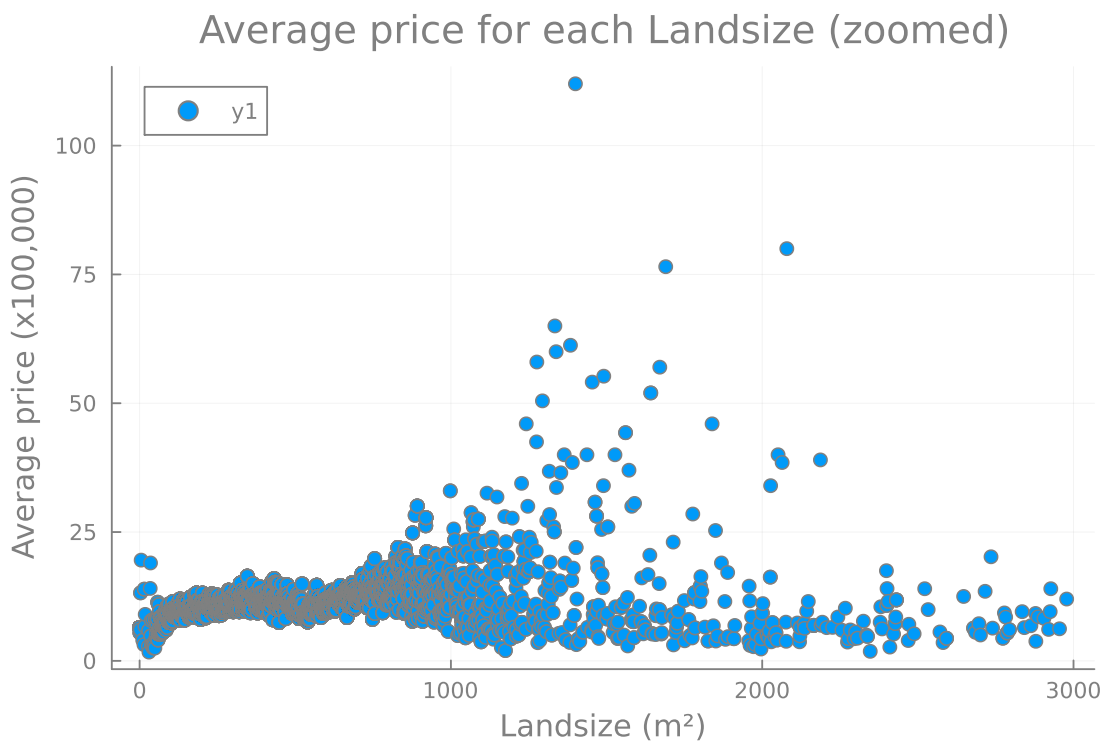
Out[19]:



This lets you see the few outliers in the data.

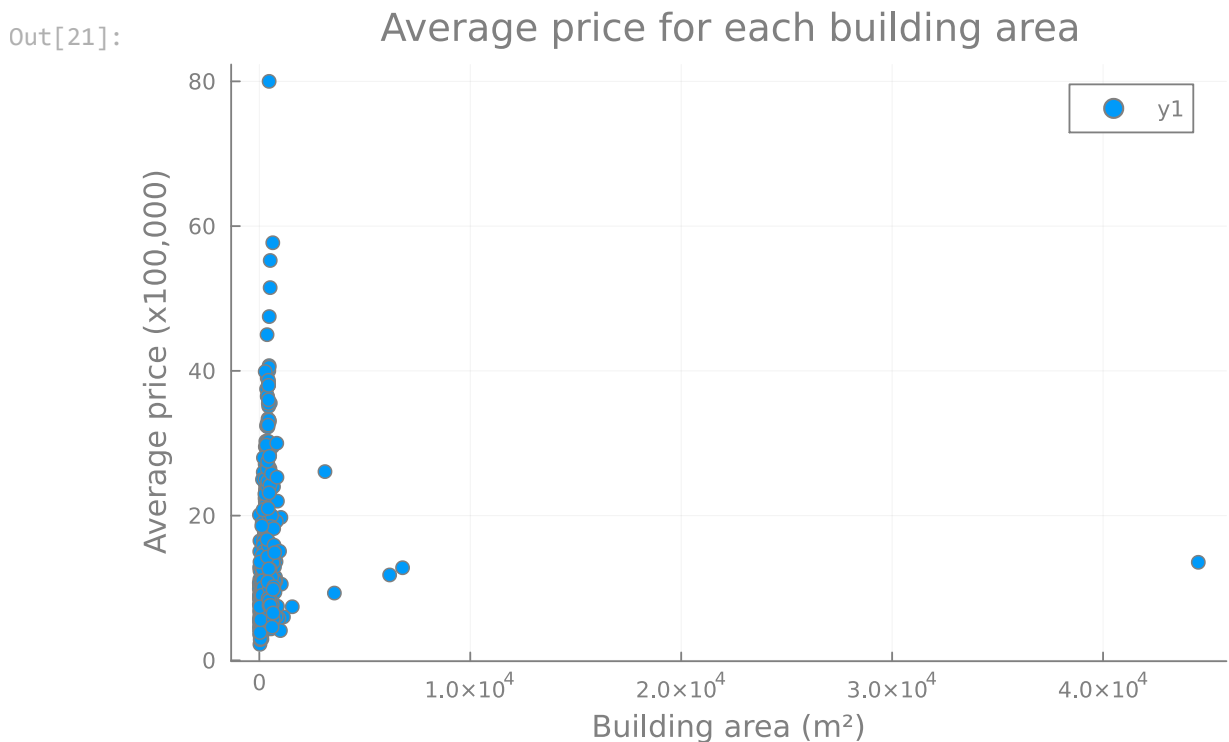
```
In [20]: # zoom_data = filter((x) -> x.< 3000, unique_vec[2])
data = select(df, [:Price, :Landsize])
data = subset(data, :Landsize => ByRow(<=(3000)); skipmissing=true)
data = groupby(data, :Landsize)
data = combine(data, :Price => mean ∘ skipmissing => :Price, :Landsize => x -> x
scatter(data.Landsize, data.Price./10^5,
title = "Average price for each Landsize (zoomed)",
xlabel = "Landsize (m²)",
ylabel = "Average price (x100,000)")
```

Out[20]:



A more useful look at the landsize, the data looks like it could be approximated as linear due to the almost constant average price. Although a logarithmic relation may be more accurate to account for the few lower prices near the origin and higher values emerging at 1000 meters squared. I was expecting a more defined increase in the data.

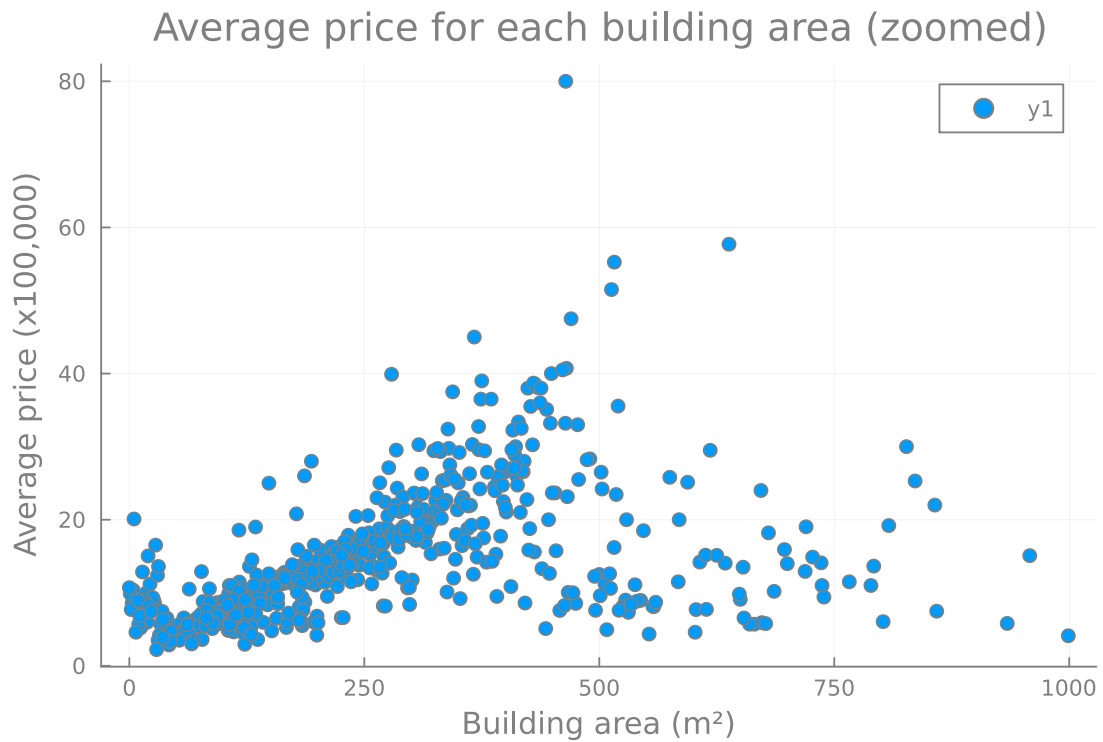
```
In [21]: #building size
data = select(df, [:Price, :BuildingArea])
data = groupby(data, :BuildingArea)
data = combine(data, :Price => mean ◦ skipmissing => :Price)
scatter(data.BuildingArea, data.Price./10^5,
title = "Average price for each building area",
xlabel = "Building area (m²)",
ylabel = "Average price (x100,000)")
```



Once again the dataset has to be narrowed to see useful information.

```
In [22]: data = select(df, [:Price, :BuildingArea])
data = subset(data, :BuildingArea => ByRow(<=(1000)); skipmissing=true)
data = groupby(data, :BuildingArea)
data = combine(data, :Price => mean ◦ skipmissing => :Price)
scatter(data.BuildingArea, data.Price./10^5,
title = "Average price for each building area (zoomed)",
xlabel = "Building area (m²)",
ylabel = "Average price (x100,000)")
#
```

Out[22]:



As expected larger buildings are more expensive. However after 450, the average price appears to be approximately constant at lower prices ~\$1,000,000.

In [23]:

```
data = select(df, [:Rooms, :Car, :Bathroom, :Price, :Bedroom2])
sort!(data)
data = coalesce.(data, -1)
disallowmissing!(data)

sum_rooms = []
using BenchmarkTools
for (i,n) in enumerate(data.Price)
    room_count = (data.Rooms[i] != -1 ? data.Bedroom2[i] : data.Rooms[i])
    if n != -1 && room_count != -1 && data.Bathroom[i] != -1 && data.Car[i] != -1
        rooms = room_count + data.Bathroom[i] + data.Car[i]
        append!(sum_rooms, rooms)
    end
end

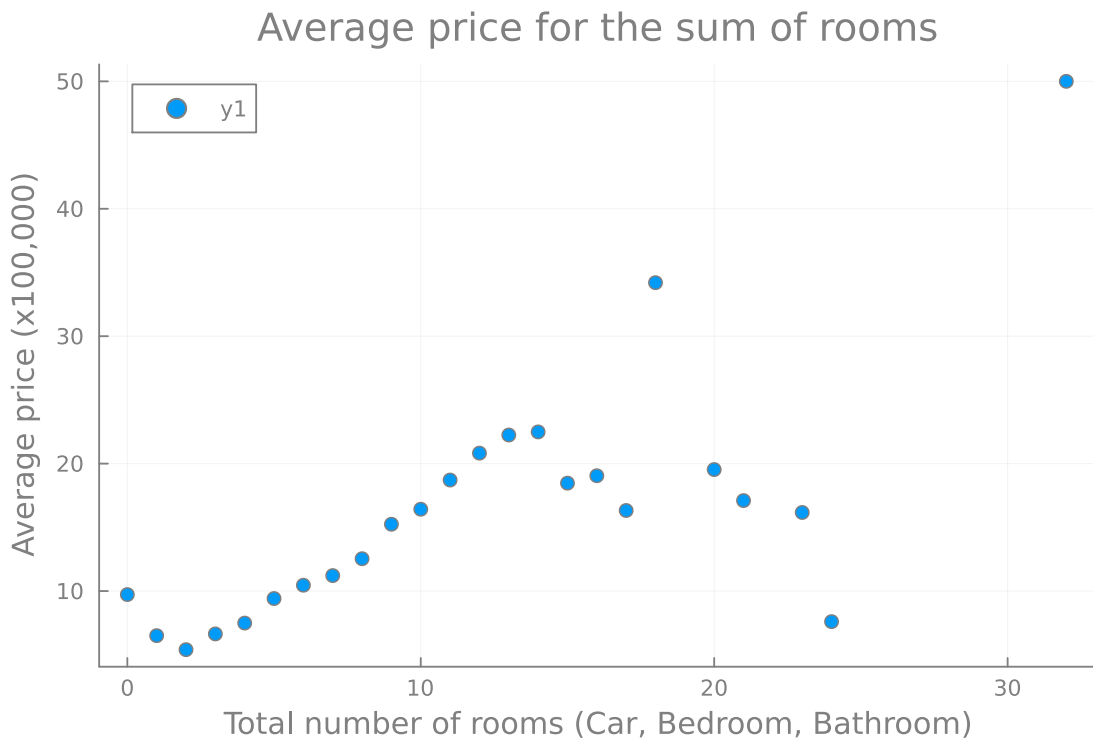
sum_rooms_prices = zeros(length(unique(sum_rooms)))
count_vec = zeros(length(unique(sum_rooms)))

for (j, val) in enumerate(unique(sum_rooms))
    for (i, n) in enumerate(data.Price)
        room_count = (data.Rooms[i] != -1 ? data.Bedroom2[i] : data.Rooms[i])
        if (n != -1 && room_count != -1 && data.Bathroom[i] != -1 && data.Car[i] != -1)
            sum_rooms_prices[j] += n
            count_vec[j] += 1
        end
    end
end

groupcount(sum_rooms)
scatter(unique(sum_rooms), (sum_rooms_prices ./ count_vec)./10^5,
title = "Average price for the sum of rooms",
xlabel = "Total number of rooms (Car, Bedroom, Bathroom)",
```

```
ylabel = "Average price (x100,000)"
#
```

Out[23]:



This relationship appears linear up to around the 15 point. Perhaps there are missing values or poor condition properties with higher room counts that bring the data down. However if we consider the 7 data points that uncharacteristically drop in price, then the linear relationship appears to be maintained.

```
In [24]: #land, build
data = select(df, [:Price, :BuildingArea, :Landsize])
data = coalesce.(data, -1)
disallowmissing!(data)

heat_land = collect(data.Landsize)
for (i, n) in enumerate(heat_land)
    if n == -1
        continue
    else
        heat_land[i] = round((n + 50) / 100) * 100
    end
end

heat_build = collect(data.BuildingArea)
for (i, n) in enumerate(heat_build)
    if n == -1
        continue
    else
        heat_build[i] = round((n + 25) / 50) * 50
    end
end

heat_land = sort(unique(heat_land))
heat_build = sort(unique(heat_build))

heat_price = zeros(length(heat_land), length(heat_build))
```

```

prices_mat = zeros(length(heat_land), length(heat_build))
price_count_mat = 2 .* ones(length(heat_land), length(heat_build))

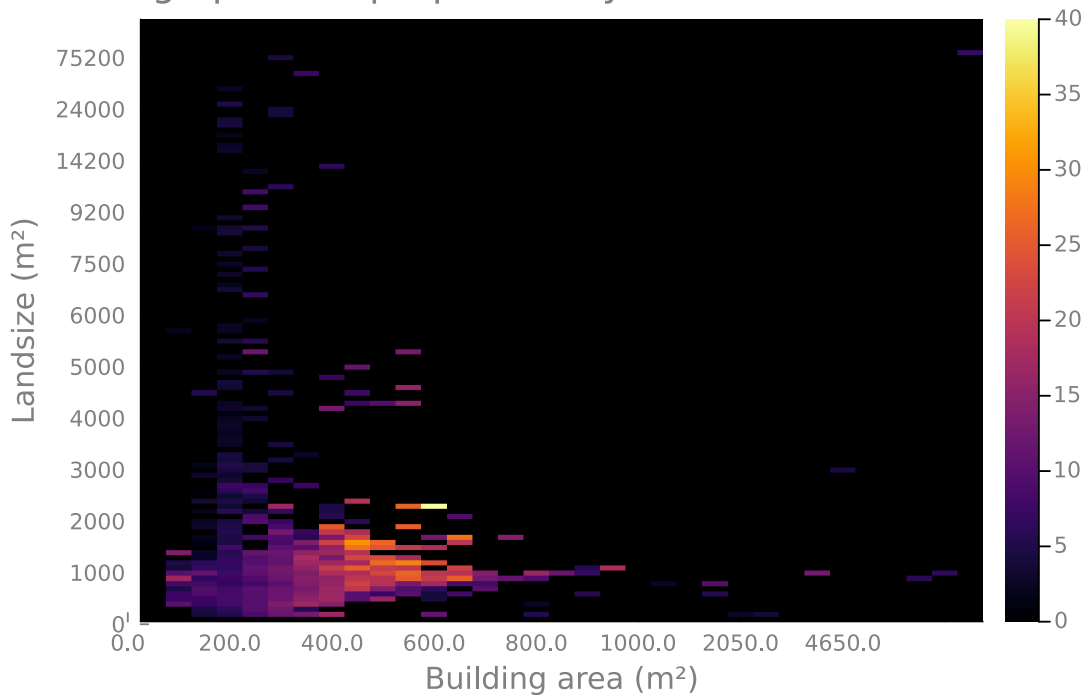
function getposition(land::Int64, build::Float64)
    land = round((land + 50) / 100) * 100
    l_index = 0
    build = round((build + 25) / 50) * 50
    b_index = 0
    for (index, val) in enumerate(heat_land)
        if land == val
            l_index = index
        end
    end
    for (index, val) in enumerate(heat_build)
        if build == val
            b_index = index
        end
    end
    return [l_index, b_index]
end

# not_neg = 0
for (index, pric) in enumerate(data.Price)
    if pric != -1 && data.Landsize[index] != -1 && data.BuildingArea[index] != -
        # not_neg += 1
        position = getposition(data.Landsize[index], data.BuildingArea[index])
        prices_mat[position[1], position[2]] += pric
        price_count_mat[position[1], position[2]] += 1
    end
    # if mod(index, 1000) == 0
    #     println(".")
    # end
end

heat_price = (prices_mat ./ (price_count_mat .- 1)) ./ 10^5
heatmap(heat_price,
title = "Average price of properties by build area and landsize",
xlabel = "Building area (m²)",
ylabel = "Landsize (m²)",
xticks = (0:4:33, heat_build[2:4:end]),
yticks = (0:10:117, heat_land[2:10:end]))

```

Out[24]: Average price of properties by build area and landsize



There is clearly an area where the building area and landsize is optimal. This lies around the area of building area = [400, 600] and landsize = [500, 2500] which is approximately the range of a standard suburban house*. As expected as you journey away from the origin the prices decrease as these properties would not be in high demand, and there is more likely to be missing data as prices are not always disclosed.

*(Taylor, J. (2022, May 4). Average House Size Australia (m2). Architecture & Design. <https://www.architectureanddesign.com.au/features/list/house-sizing-australia-trends-averages-and-standar>)

```
In [25]: #Land, distance
data = select(df, [:Price, :Distance, :Landsize])
heat_dis = collect(skipmissing(data.Distance))

data = coalesce.(data, -1)
disallowmissing!(data)

heat_land = collect(data.Landsize)
for (i, n) in enumerate(heat_land)
    if n == -1
        continue
    else
        heat_land[i] = round((n + 50) / 100) * 100
    end
end

# for (i, n) in enumerate(heat_dis)
#     heat_dis[i] = round((n + 25) / 50) * 50
# end

heat_land = sort(unique(heat_land))
heat_dis = sort(unique(heat_dis))
```

```

heat_price = zeros(length(heat_land), length(heat_dis))
prices_mat = zeros(length(heat_land), length(heat_dis))
price_count_mat = 2 .* ones(length(heat_land), length(heat_dis))

function getposition(land::Int64, distance::Float64)
    land = round((land + 50) / 100) * 100
    l_index = 0
    # distance = round((distance + 50) / 100) * 100
    d_index = 0
    for (index, val) in enumerate(heat_land)
        if land == val
            l_index = index
        end
    end
    for (index, val) in enumerate(heat_dis)
        if distance == val
            d_index = index
        end
    end
    return [l_index, d_index]
end

# not_neg = 0
for (index, pric) in enumerate(data.Price)
    if pric != -1 && data.Landsize[index] != -1 && data.Distance[index] != -1
        # not_neg += 1
        position = getposition(data.Landsize[index], data.Distance[index])
        prices_mat[position[1], position[2]] += pric
        price_count_mat[position[1], position[2]] += 1
    end
    # if mod(index, 1000) == 0
    #     println(".")
    # end
end

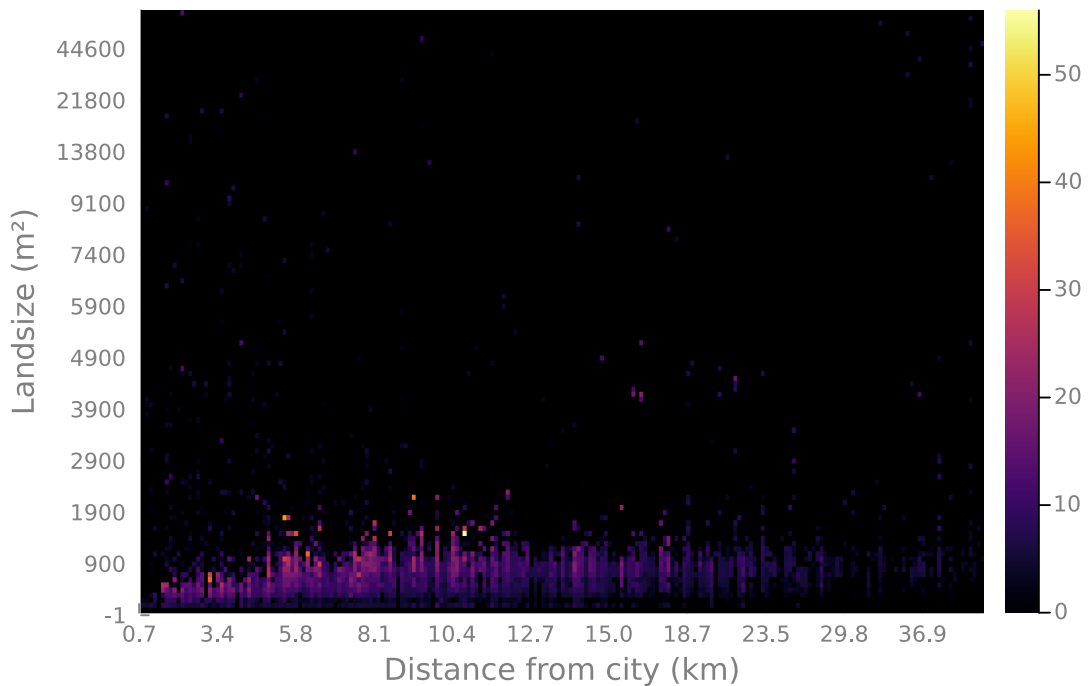
prices_mat

heat_price = (prices_mat ./ (price_count_mat .- 1)) ./ 10^5

heatmap(heat_price,
title = "Average price of properties by landsize and distance from city",
xlabel = "Distance from city (km)",
ylabel = "Landsize (m²)",
xticks = (0:20:215, heat_dis[2:20:end]),
yticks = (0:10:117, heat_land[1:10:end]))

```

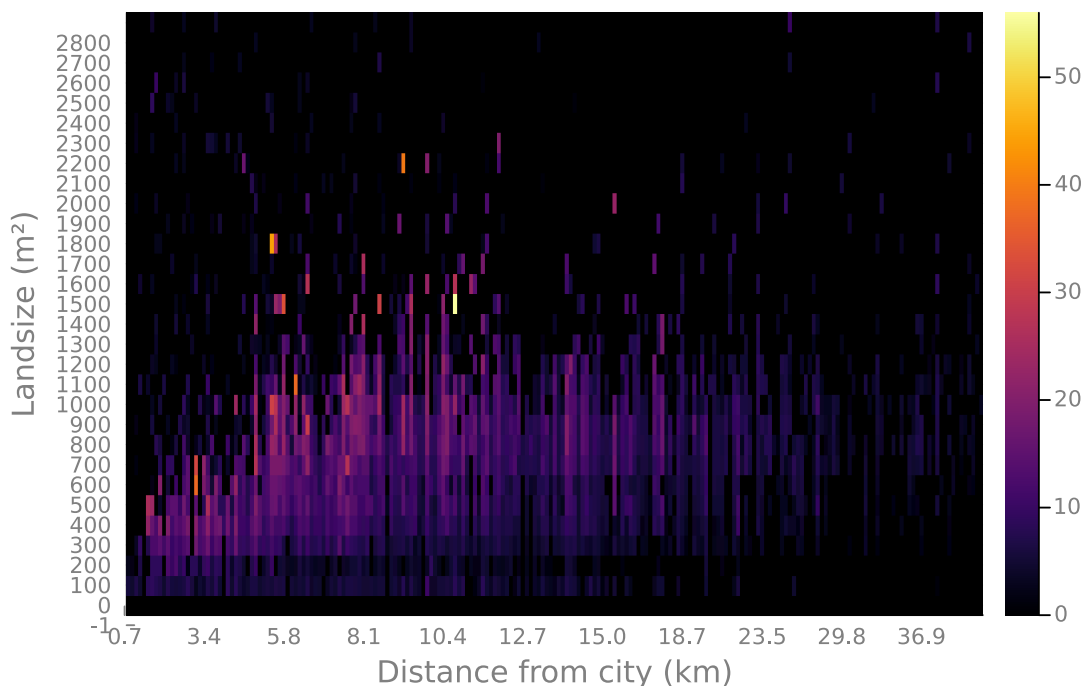
Out[25]: Average price of properties by landsize and distance from ci



The price is related to both distance and landsize as when the landsize increases and the distance decreases, on average, the price decreases. There does seem to be the most missing data in this heat map compared to the others.

```
In [26]: heatmap(heat_price[1:30,:],
title = "Average price of properties by landsize and distance from city",
xlabel = "Distance from city (km)",
ylabel = "Landsize (m²)",
xticks = (0:20:215, heat_dis[2:20:end]),
yticks = (0:30, heat_land[1:30]))
#
```

Out[26]: Average price of properties by landsize and distance from ci



Zooming in on the area where there is more data, we can see distance changes the price more than landsize does when observing the rate of change of tick marks rather than actual values. There is almost a line of optimisation for price going from just above the origin to $\sim(11,1600)$ where there is a lack of data to the left of the line and decrease in average price to the right.

```
In [27]: #build, distance
data = select(df, [:Price, :BuildingArea, :Distance])

heat_distance = collect(skipmissing(data.Distance))

data = coalesce.(data, -1)
disallowmissing!(data)

heat_build = collect(data.BuildingArea)
for (i, n) in enumerate(heat_build)
    if n == -1
        continue
    else
        heat_build[i] = round((n + 25) / 50) * 50
    end
end

heat_distance = sort(unique(heat_distance))
heat_build = sort(unique(heat_build))

heat_price = zeros(length(heat_build), length(heat_distance))
prices_mat = zeros(length(heat_build), length(heat_distance))
price_count_mat = 2 .* ones(length(heat_build), length(heat_distance))

function getposition(build::Float64, distance::Float64)
    d_index = 0
    build = round((build + 25) / 50) * 50
    b_index = 0
    for (index, val) in enumerate(heat_distance)
        if distance == val
            d_index = index
        end
    end
    for (index, val) in enumerate(heat_build)
        if build == val
            b_index = index
        end
    end
    return [b_index, d_index]
end

# not_neg = 0
for (index, pric) in enumerate(data.Price)
    if pric != -1 && data.Distance[index] != -1 && data.BuildingArea[index] != -
        # not_neg += 1
        position = getposition(data.BuildingArea[index], data.Distance[index])
        prices_mat[position[1], position[2]] += pric
        price_count_mat[position[1], position[2]] += 1
    end
    # if mod(index, 1000) == 0
    #     println(".")
end
```

```

# end
end

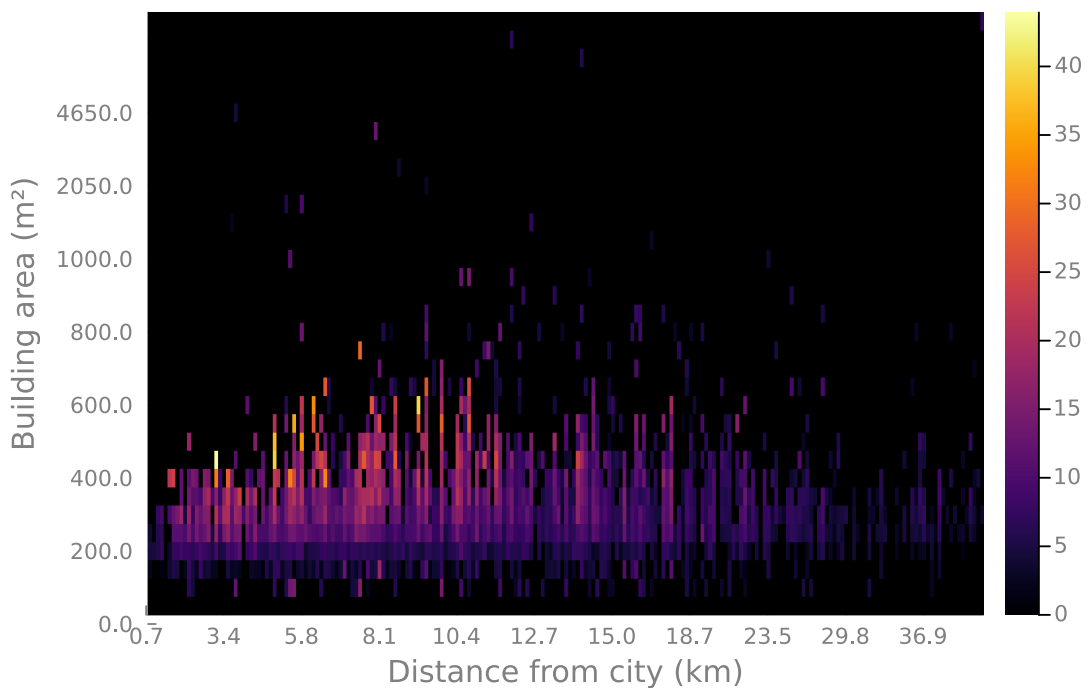
heat_price = (prices_mat ./ (price_count_mat .- 1)) ./ 10^5

heatmap(heat_price,
title = "Average price of properties by Build area and distance from city",
xlabel = "Distance from city (km)",
ylabel = "Building area (m²)",
xticks = (0:20:215, heat_dis[2:20:end]),
yticks = (0:4:33, heat_build[2:4:end]))

#

```

Out[27]: Average price of properties by Build area and distance from city



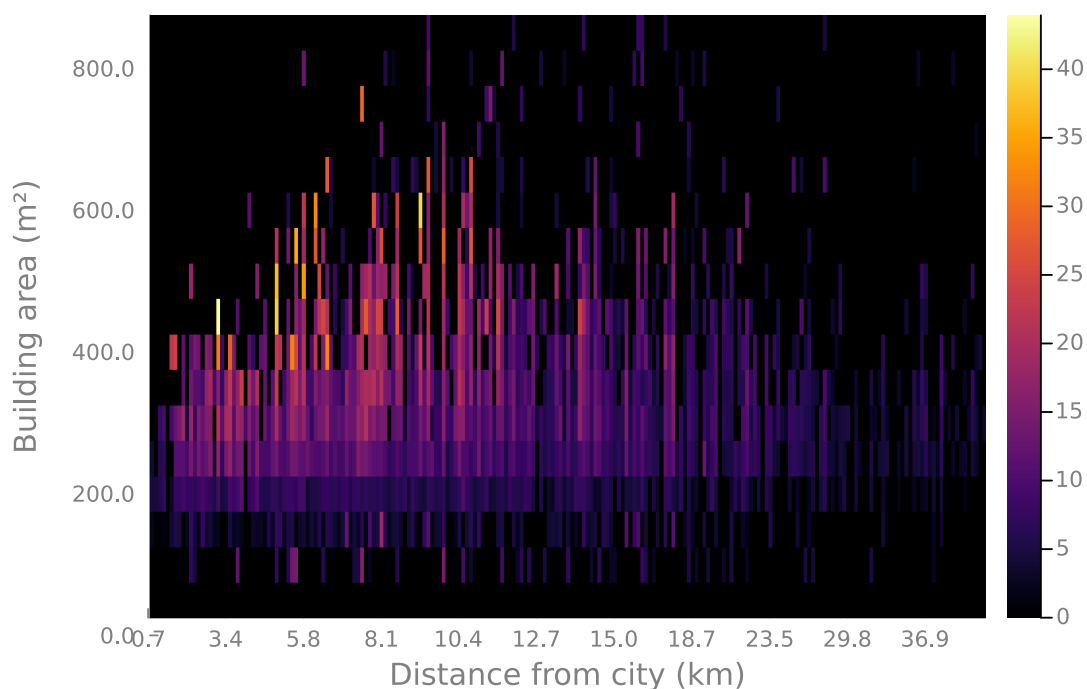
Once again building size appears to have more of an impact on the pricing of properties.

```

In [28]: heatmap(heat_price[1:17, :],
title = "Average price of properties by Build area and distance from city",
xlabel = "Distance from city (km)",
ylabel = "Building area (m²)",
xticks = (0:20:215, heat_dis[2:20:end]),
yticks = (0:4:33, heat_build[2:4:end]))

```

Out[28]: average price of properties by Build area and distance from city



The region of high prices and the way it diminishes appears more radial than the other heat maps.

Task 3

plots that help visualise trends in data. volume of sales (num props and values) over time.
also type h over time. time in months

```
In [29]: volume_data = select(df, [:Date, :Price, :Type])

equal_times(date1::Date, date2::Date) = Dates.yearmonth(date1) == Dates.yearmonth(date2)

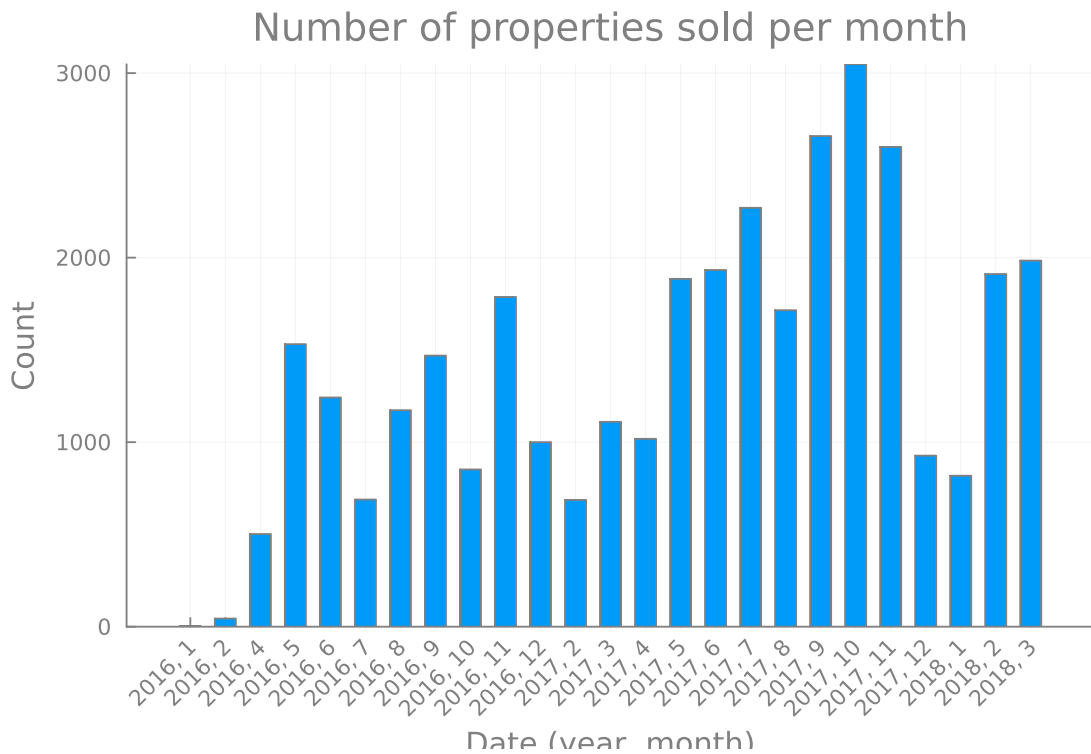
transform!(volume_data, :Date => x -> yearmonth.(x))

volume_data = groupby(volume_data, :Date_function)

#num of sales
data = combine(volume_data, nrow => :count)
sort!(data)
data = transform(data, :Date_function => ByRow(x -> "$(x[1])"*", "*"$(x[2])") =>

bar(data.Date, data.count,
xticks=(0.5:24.5, data.Date),
xrotation = 45,
bar_width = 0.6,
widen = true,
legend = false,
title = "Number of properties sold per month",
xlabel = "Date (year, month)",
ylabel = "Count")
# size=(1000, 500))
#
```

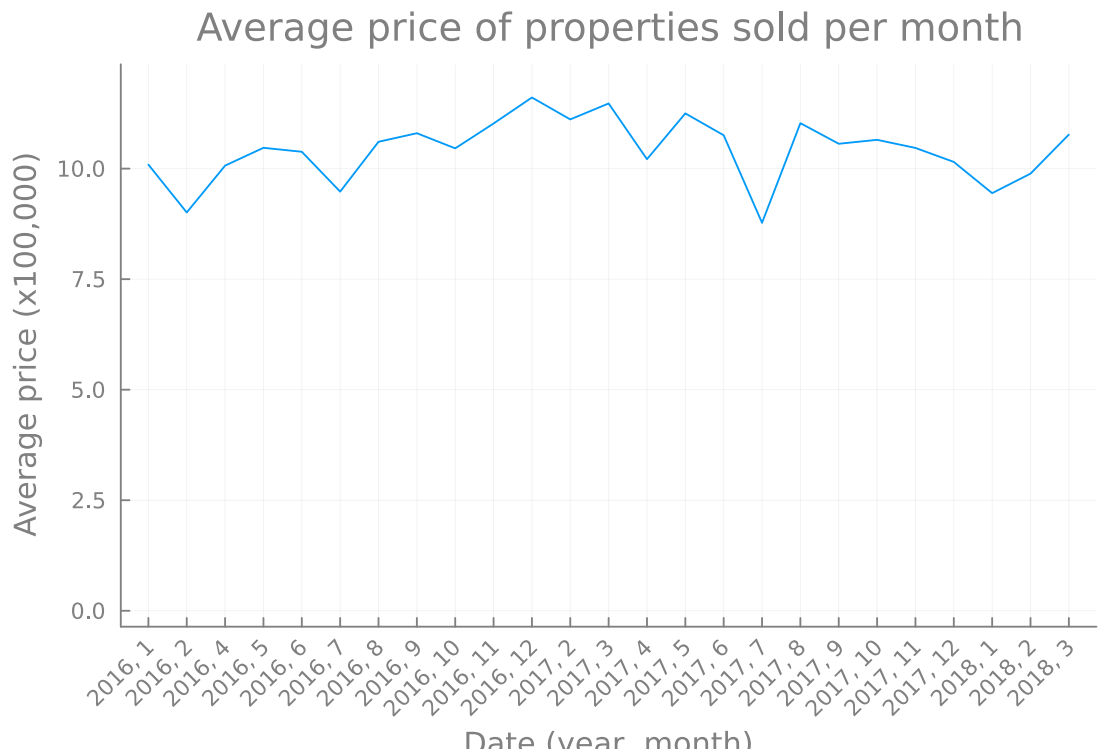
Out[29]:



I suspect there is missing data in the early years as the numbers low. There appears to be a periodic relationship as the prices rise and fall frequently with a suprising increase in 2017. Although as some months are missing there will be gaps and hence inaccuracy in the approximated relationship.

```
In [30]: #price of sales
# passmissing(select(df, [:Date, :Price]))
data = combine(volume_data, :Price => mean ◦ skipmissing => :Price)
sort!(data)
data = transform(data, :Date_function => ByRow(x -> "$(x[1])"*", "*"$(x[2])") =>
plot(data.Date, data.Price./10^5, ylims = [0,12],
xticks=(0.5:24.5, data.Date),
xrotation = 45,
bar_width = 0.6,
widen = true,
legend = false,
title = "Average price of properties sold per month",
xlabel = "Date (year, month)",
ylabel = "Average price (x100,000)")
```

Out[30]:



The price appears to fluctuate a bit but stays around the earlier observed mean of \$1,000,000.

```
In [31]: #type count
volume_data = select(df, [:Date, :Type])
transform!(volume_data, :Date => x -> yearmonth.(x))

volume_data = groupby(volume_data, [:Date_function, :Type])

# for each year, count groups plot the se for each year
data = combine(volume_data, nrow => :count)
# sort!(data, :Date_function)
data = transform(data, :Date_function => ByRow(x -> "$(x[1])"*", "*"$(x[2])") =>

sort!(data, [:Date_function, :Type])
prop_data = groupby(data, :Date)

props = []
for i in 1:length(prop_data)
    frame = prop_data[i]
    append!(props, frame.count[1] / sum(frame.count))
end

p1 = plot(data.Date, data.count, group = data.Type, ylims = [0, 2300],
xticks=(0.5:24.5, unique(data.Date)),
xrotation = 45,
bar_width = 0.6,
widen = true,
legend = false,
title = "Number of Types of properties sold per month",
xlabel = "Date (year, month)",
ylabel = "Count")

p2 = plot(unique(data.Date), props,
```

```
xticks=(0.5:24.5, unique(data.Date)),
xrotation = 45,
bar_width = 0.6,
widen = true,
legend = false,
title = "Proportion of houses sold per month",
xlabel = "Date (year, month)",
ylabel = "Proportion")

plot(p1, p2, layout=grid(1,2), size = (1100,500))
```

Out[31]:



There are always more house properties sold as both plots show. As plot 2 shows more than half the properties sold per month are houses except for February 2016. However, there is no immediately obvious relation between the proportion of houses across time.

Task 4

Comments marked with ## are used to explain the process as you follow the code

In [32]: `using Random, GLM`

```
##Making the train and validate data as per the practical
n, _ = size(df)
Random.seed!(0)
p_validate = 0.3
n_validate = floor{Int, p_validate*n}
validate_index_set = sample(1:n, n_validate, replace = false)
train_index_set = setdiff(1:n, validate_index_set);
n_validate = length(validate_index_set)
n_train = length(train_index_set)
n_train, n_validate
df_train = df[train_index_set,:]
df_validate = df[validate_index_set,:];

mse(y,ŷ) = sqrt(sum(skipmissing(y-ŷ).^2))
```

Out[32]: `mse (generic function with 1 method)`

In [33]: `## The natural logarithm of price was used as, through exploring different trans`
`## First I will test the individual variables with price, to match each plot from`

```

model1 = glm(@formula(log(Price) ~ Distance), df_train, Normal(), IdentityLink())
mse_1 = mse(predict(model1, df_validate), log.(df_validate.Price))

model2 = glm(@formula(log(Price) ~ Rooms), df_train, Normal(), IdentityLink())
mse_2 = mse(predict(model2, df_validate), log.(df_validate.Price))

model3 = glm(@formula(log(Price) ~ Car), df_train, Normal(), IdentityLink()) #Lc
mse_3 = mse(predict(model3, df_validate), log.(df_validate.Price))

model4 = glm(@formula(log(Price) ~ Bathroom), df_train, Normal(), IdentityLink())
mse_4 = mse(predict(model4, df_validate), log.(df_validate.Price))

model5 = glm(@formula(log(Price) ~ Landsize), df_train, Normal(), IdentityLink())
mse_5 = mse(predict(model5, df_validate), log.(df_validate.Price))

model6 = glm(@formula(log(Price) ~ BuildingArea), df_train, Normal(), IdentityLi
mse_6 = mse(predict(model6, df_validate), log.(df_validate.Price))

## All p values are < 0.05 suggesting all variables impact the pricing of a hous

## Next Looking at the combination of the continuous variables used in task 2.
model7 = glm(@formula(log(Price) ~ Distance + Landsize), df_train, Normal(), Ide
mse_7 = mse(predict(model7, df_validate), log.(df_validate.Price))
## By the p values, Landsize has less impact than distance on the error

model8 = glm(@formula(log(Price) ~ Distance + BuildingArea), df_train, Normal(),
mse_8 = mse(predict(model8, df_validate), log.(df_validate.Price))
## By the p values, Both variables impact the model by the same amount

model9 = glm(@formula(log(Price) ~ Landsize + BuildingArea), df_train, Normal(),
mse_9 = mse(predict(model9, df_validate), log.(df_validate.Price))
## By the p values, Landsize has less impact than building area on the error

model10 = glm(@formula(log(Price) ~ Car + Rooms + Bathroom), df_train, Normal(),
mse_10 = mse(predict(model10, df_validate), log.(df_validate.Price))
## By the p values, Car spots has a value of ~0.6 suggesting in this model it ha

## Once again all p values are less than 0.05 for models 7 - 9. model 10 has one

##Testing combinations of variables that I think would have an impact on price
##to better understand how combining multiple variable models will impact the mo

model11 = glm(@formula(log(Price) ~ Distance + Landsize + BuildingArea), df_trai
mse_11 = mse(predict(model11, df_validate), log.(df_validate.Price))
## p values are simmilar when testing combinations of only two of the variables.
## The mean square error is still to high for this to produce a good model.

model12 = glm(@formula(log(Price) ~ BuildingArea / (Car + Rooms + Bathroom)), df
mse_12 = mse(predict(model12, df_validate), log.(df_validate.Price))
## The building area divided by the rooms will comment on the size of the rooms.
## The p value for the combinnation of the variables is less than 0.05 and the e

model13 = glm(@formula(log(Price) ~ (BuildingArea + Landsize)/ (Car + Rooms + Ba
mse_13 = mse(predict(model13, df_validate), log.(df_validate.Price))

model14 = glm(@formula(log(Price) ~ Distance + (BuildingArea / (Car + Rooms + B
mse_14 = mse(predict(model14, df_validate), log.(df_validate.Price))

```

```

## Model 13 and 14 were extensions on model 12. These led to higher errors so th
##To do so I observed a model where building area is added not a numerator

model15 = glm(@formula(log(Price) ~ BuildingArea + Car + Rooms + Bathroom), df_t
mse_15 = mse(predict(model15, df_validate), log.(df_validate.Price))
## This produced a better model than model 12. Once again the car variable has a

model16 = glm(@formula(log(Price) ~ BuildingArea + Rooms + Bathroom), df_train,
mse_16 = mse(predict(model16, df_validate), log.(df_validate.Price))
## Testing the impact of not including the car variable does increase the error

model17 = glm(@formula(log(Price) ~ Landsize + Distance + BuildingArea + Car +
mse_17 = mse(predict(model17, df_validate), log.(df_validate.Price));
## Simply adding the variables produces a improved model with all variables havi

# For the approximation of the relationship of log(price) and the room related v

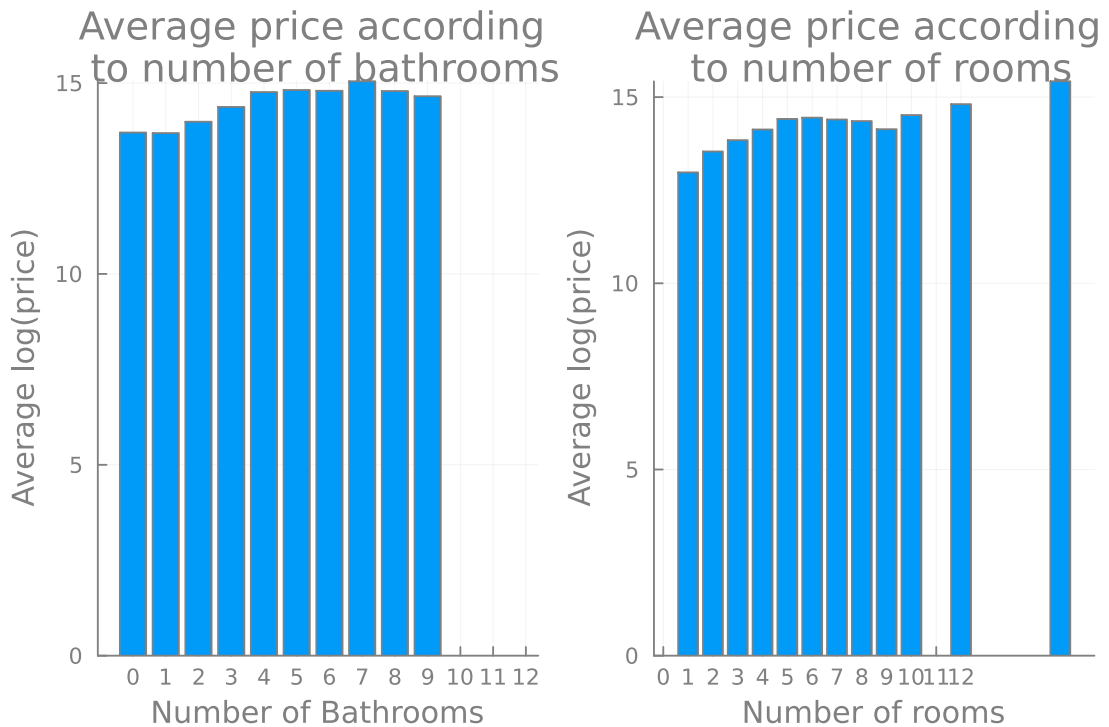
data = select(df, [:Price, :Bathroom])
data = groupby(data, :Bathroom)
data = combine(data, :Price => mean ∘ skipmissing => :Price)
plot_1 = bar(data.Bathroom, log.(data.Price),
title = "Average price according \n to number of bathrooms",
xlabel = "Number of Bathrooms",
ylabel = "Average log(price)",
xticks = 0:1:12,
legend = false)

data = select(df, [:Price, :Rooms])
data = groupby(data, :Rooms)
data = combine(data, :Price => mean ∘ skipmissing => :Price)
plot_2 = bar(data.Rooms, log.(data.Price),
title = "Average price according \n to number of rooms",
xlabel = "Number of rooms",
ylabel = "Average log(price)",
xticks = 0:1:12,
legend = false)

plot(plot_1, plot_2)

```

Out[33]:



Having found the best way to combine the variables, I went back and observed the relationships in task 2 and took into account what I have found in these models.

The variables Car, Rooms and bathroom appear to produce a linear relation ship, so any transformation applied to one of these will be applied to the others. As the car variable has little impact on the mean square error (MSE), any transformation can be applied to it. Bedrooms and bathrooms could be approximated as a root function, as seen in the plot above, so the square root will be taken of these variables, and also the car variable.

The distance and landsize appear linear in the heat map, however more model testing supported this when trying to decrease the MSE.

The building area appears to be either linear or some sort of polynomial. By approximating it as linear, quadratic, cubic and quartic the quadratic model had the lowest MSE. As the eponent increased after 2, the MSE began to increae, hence higher exponents were not tested.

The inclusion of these approximations led to the final model below.

```
In [34]: model18 = glm(@formula(log(Price) ~ Landsize + Distance + BuildingArea^2 + (sqrt
mse_18 = mse(predict(model18, df_validate), log.(df_validate.Price))

# Putting the model data (p values, MSE, etc.) in a dataframe for easier observa
models = [model11, model12, model13, model14, model15, model16, model17, model18, model19
mses = [mse_1, mse_2, mse_3, mse_4, mse_5, mse_6, mse_7, mse_8, mse_9, mse_10, m
p_vals = [coefstable(model).cols[4] for model in models]
model_nums = ["1", "2", "3", "4", "5", "6", "7", "8", "9", "10", "11", "12", "13
accuracy = DataFrame(Model = model_nums, P = p_vals, MSE = mses)

# Allows us to see the order of models from best to worst.
sort!(accuracy, :MSE)

# Applying the square root of the MSE and the normalised values for alternate in
```

```
transform!(accuracy, :MSE => ByRow(x -> sqrt(x)) => :RMSE)
transform!(accuracy, :RMSE => ByRow(x -> x/(maximum(skipmissing(log.(df_validate
transform!(accuracy, :MSE => ByRow(x -> x/(maximum(skipmissing(log.(df_validate.
```

Out[34]: 18×6 DataFrame

Row	Model	P	MSE	RMSE	Norm_RMSE	Norm_MSE
	String	Array...	Float64	Float64	Float64	Float64
1	18	[0.0, 8.13016e-8, 0.0, 0.50919, 0.0359795, 0.0, 2.04781e-50]	22.0365	4.69431	1.24238	5.8321
2	17	[0.0, 3.06054e-6, 0.0, 1.01676e-23, 1.20355e-9, 0.0, 2.83209e-39]	26.0029	5.09931	1.34956	6.88183
3	13	[0.0, 0.21786]	28.6523	5.35278	1.41665	7.583
4	15	[0.0, 5.23877e-24, 0.185081, 1.07082e-168, 8.67623e-53]	30.9027	5.55902	1.47123	8.17857
5	16	[0.0, 1.75004e-23, 6.93833e-176, 1.52804e-51]	30.9772	5.56571	1.473	8.19829
6	14	[0.0, 2.29437e-117, 2.48281e-46]	32.0133	5.65803	1.49743	8.47251
7	12	[0.0, 6.90425e-44]	33.2432	5.76569	1.52592	8.79801
8	10	[0.0, 0.591981, 0.0, 2.21259e-94]	34.4267	5.86743	1.55285	9.11124
9	4	[0.0, 0.0]	37.1263	6.09313	1.61258	9.82568
10	7	[0.0, 1.28105e-158, 3.24496e-6]	37.2287	6.10153	1.61481	9.85279
11	5	[0.0, 0.000316279]	38.1937	6.18011	1.6356	10.1082
12	3	[0.0, 9.04468e-150]	39.4941	6.28444	1.66321	10.4524
13	2	[0.0, 0.0]	39.7742	6.30668	1.6691	10.5265
14	1	[0.0, 3.3991e-194]	45.9192	6.77637	1.79341	12.1528
15	9	[0.0, 0.00531841, 5.8638e-140]	55.1363	7.42538	1.96517	14.5922
16	11	[0.0, 1.43212e-139, 7.95928e-10, 1.20295e-177]	57.9068	7.60965	2.01394	15.3254
17	6	[0.0, 1.29384e-172]	58.5961	7.65481	2.02589	15.5078
18	8	[0.0, 1.01832e-172, 4.96532e-223]	61.1656	7.82084	2.06983	16.1878

In [35]: `plot1 = scatter(predict(model18, df_validate), log.(df_validate.Price), xlims = xlabel="Pedicted log(Price)", ylabel="Recorded log(Price)", title="Best model (18)",`

```

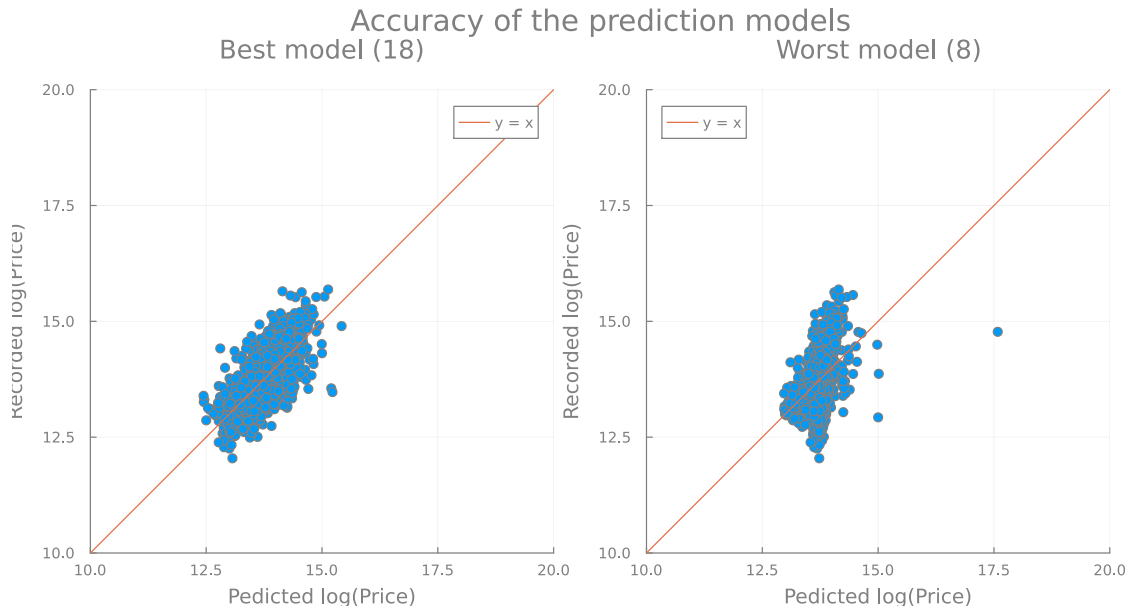
label = false)
plot!(plot1, [10,20], [10,20], label="y = x")

plot2 = scatter(predict(model8, df_validate), log.(df_validate.Price), xlims = [
xlabel="Predicted log(Price)",
ylabel="Recorded log(Price)",
title="Worst model (8)",
label = false)
plot!(plot2, [10,20], [10,20], label="y = x")

plot(plot1, plot2, plot_title = "Accuracy of the prediction models", size = (900

```

Out[35]:



Comparing the best and worst models, it is clear that the best model does follow the line $y=x$, the perfect prediction line, suggesting that it can predict house prices to some level of accuracy. However the normalised error values are still quite large which is reflected in the clumping of the data points. As seen from the data frame, the addition of all variables does create a better model.

Task 5

Should I run out of time to complete this task, the following is a description of process I would do to make the animation.

- Get a picture of the Melbourne area which contains lines of latitude and longitude containing the minimum and maximum values in the data.
- Mark the location of the Central Business district (CBD) as this is used in the distance column of the data.
- Approximate properties by their postcode and/or council area.
- Mark the number of sales by using circles where the size of the circle represents the number of sales.
- Use a colour scale for the circles to represent the average price of sales.
- On the side, present the proportion of sales that are houses for each area.
- Also show the average of the sum of the land size and building area of each location.
- time will progress in months.

```

In [36]: ##Code commented so it does not run and make the file too large
## I used this video as a guide to making an animation on makie
# https://www.youtube.com/watch?v=L-gyDvhjzGQ

#= <= Just remove this to run
using Pkg
Pkg.activate(".")
using CSV, DataFrames, HTTP, CSV, StatsBase, Dates
path = "./data/Melbourne_housing_FULL.csv"
old_df = CSV.read(path, DataFrame)

df = copy(old_df)

allowmissing!(df, :Distance)
for (i,n) in enumerate(df.Distance)
    if n == "#N/A"
        df.Distance[i] = missing
    end
end
df.Distance .= passmissing(parse).(Float64, df.Distance)

count = 0
for (index, val) in enumerate(skipmissing(df.Landsize))
    if val == 0 && (df.Type == "h" || df.Type == "t" || df.Type == "o res")
        count += 1
    end
end

df.YearBuilt .= coalesce.(df.YearBuilt, -1)

# df.Lattitude .= coalesce.(df.Lattitude, 0.0)
# df.Longitude .= coalesce.(df.Longitude, 0.0)

allowmissing!(df, :CouncilArea)
allowmissing!(df, :Postcode)
allowmissing!(df, :Regionname)
allowmissing!(df, :Propertycount)

# unique(df.CouncilArea) #missing as #N/A => changing based on suburb/address or
function findarea(suburb::String31)
    for (i,n) in enumerate(df.Suburb)
        if n == suburb && df.CouncilArea[i] != "#N/A"
            return df.CouncilArea[i]
        end
    end
    return missing
end

for (i,n) in enumerate(df.CouncilArea)
    if n == "#N/A" && !(df.Suburb[i] isa Nothing)
        df.CouncilArea[i] = findarea(df.Suburb[i])
    end
end

# unique(df.Postcode) #missing as #N/A => changing based on suburb/address

```

```

function findcode(suburb::String31)
    for (i,n) in enumerate(df.Suburb)
        if n == suburb && df.Postcode[i] != "#N/A"
            return df.Postcode[i]
        end
    end
    return missing
end

for (i,n) in enumerate(df.Postcode)
    if n == "#N/A"
        df.Postcode[i] = findcode(df.Suburb[i])
    end
end

# unique(df.Regionname) #missing as #N/A => changing based on suburb/address
function findregion(suburb::String31)
    for (i,n) in enumerate(df.Suburb)
        if n == "suburb" && df.Regionname[i] != "#N/A"
            return df.Regionname[i]
        end
    end
    return missing
end

for (i,n) in enumerate(df.Regionname)
    if n == "#N/A"
        df.Regionname[i] = findregion(df.Suburb[i])
    end
end

# unique(df.Propertycount) #missing as #N/A => changing based on suburb/address
function findcount(suburb::String31)
    for (i,n) in enumerate(df.Suburb)
        if n == "suburb" && df.Propertycount[i] != "#N/A"
            return df.Propertycount[i]
        end
    end
    return missing
end

for (i,n) in enumerate(df.Propertycount)
    if n == "#N/A"
        df.Propertycount[i] = findcount(df.Suburb[i])
    end
end

df.Propertycount = passmissing(parse).(Int64, df.Propertycount)
df.Postcode .= passmissing(parse).(Int64, df.Postcode)

# turns Date string into a form that can be turned into a Date type
function parsedate(date)
    result = ""
    for n in date
        if n == '/'
            result *= "-"
        else
            result *= n
        end
    end
    return result
end

```

```

end

#Creates the Date column to be a Date type
insertcols!(df, :newDate => Date("0001-01-01"))
for (i,n) in enumerate(df.Date)
    date = String(parsedate(n))
    df.newDate[i] = Date(date, "d-m-y")
end
df = select!(df, Not(:Date))
rename!(df, :newDate => :Date)

insertcols!(df, :newmeth => "")
for (i,n) in enumerate(df.Method)
    method = String(n)
    df.newmeth[i] = method
end
df = select!(df, Not(:Method))
rename!(df, :newmeth => :Method)

#####
#Get data to use
data = transform(df, :Date => x -> yearmonth.(x))
data = transform(data, :Date_function => ByRow(x -> "$(x[1])"*", "*"$(x[2])") =>
data_grouped = groupby(sort!(data, :Date_function), [:Date_function, :Postcode])

#CBD Location supplied here: https://mapcarta.com/Melbourne/CBD#:~:text=Longitude
#Lat = -37.8136, Long = 144.963

#Find Location for each Postcode
Locations = []
post_loc = groupby(data, :Postcode)
post_loc = combine(post_loc, :Longitude => mean ∘ skipmissing => :Longitude_mean,
transform!(post_loc, [:Longitude_mean, :Latitude_mean] => ByRow((x,y) -> (x, y))
sort!(post_loc, [:Longitude_mean, :Latitude_mean])
pop!(post_loc) # remove missing postcode row
pop!(post_loc) # remove row with non recorded latitude and longitude of postcode

#Get domain and range of plot
# maximum(skipmissing(old_df.Longitude)) # - 145° 31' 34.86"
# minimum(skipmissing(old_df.Longitude)) #- 144° 25' 25.64"

# maximum(skipmissing(old_df.Lattitude)) # - -37° 23' 24.72"
# minimum(skipmissing(old_df.Lattitude)) # - -38° 11' 25.55"

#initialise plot
using CairoMakie, FileIO

sales_data = combine(data_grouped, nrow => :Count,
:Price => mean ∘ skipmissing => :Price_mean,
:Longitude => mean ∘ skipmissing => :Longitude_mean,
:Lattitude => mean ∘ skipmissing => :Latitude_mean)

function next_markers(index::Int64)
    return groupby(sales_data, :Date_function)[index]
end

function getsizes(data)::Vector{Float64}
    result = zeros(210)

```

```

# @show Length(unique(data.Count))
vals = sort!(unique(data.Count))
sizes = 2:(20 - 2)/Length(unique(data.Count)):20
# transform!(data, :Count => ByRow(x -> round((x + 2.5)/5) * 5))
for (i, code) in enumerate(skipmissing(data.Postcode))
    for (index, post) in enumerate(post_Loc.Postcode)
        if code == post
            for (size, val) in enumerate(vals)
                if val == data.Count[i]
                    result[index] = sizes[size]
                end
            end
        end
    end
end
end
return result
end

frames = 1:Length(unique(price_data.Date_function))
frame_rate = 1

mark_size = Observable(zeros(210))
date_o = Observable("string")

fig = Figure(); display(fig)
img = Load("Images/Melbourne_nogrid.png")
land = Axis(fig[1,1], title = date_o, xlabel = "Longitude", ylabel="Latitude", L
image!(land, (144.4, 145.5), (-38.2, -37.39), rotr90(img), alpha=0.5)
scatter!(144.963, -37.8136, marker = :star4, color=:red)
Colorbar(fig[1,2], colormap = :thermal, highclip=:cyan, lowclip=:red)
scatter!(land, post_Loc.Longitude_mean, post_Loc.Latitude_mean, markersize = mar

Locations_by_post = post_Loc.Postcode
function animstep!(mark_size, index::Int64)
    new_data = next_markers(index)
    mark_size[] = getsizes(new_data)
    date_o[] = "Number of properties sold. Date: " * unique(data.Date)[index]
end

# index = 25
# for i in 1:1
#     index += 1
#     @show index
#     animstep!(mark_size, index)
#     display(fig)
#     sleep(0.001)

# end

if true
index = 0
count = 0
frames = 1:250
record(fig, "task5.mp4", frames; frame_rate = 1) do i
    global count += 1
    if mod(count, 10) == 0
        global index += 1
        # @show index
        animstep!(mark_size, index)
        sleep(0.01)
    end
end

```

```
    end  
end  
end  
  
#color range to use for prices  
# range(HSV(0,1,1), stop=HSV(-90,1,1))  
=# #<= Remove this line
```